OPERATING SYSTEMS

ASSIGNMENT 1 - CO1

Evaluate and Justify the Operating-System Design of a Smart Hospital

Name: B Ashvin

RegisterNumber:192511235

Course: Operating System

Course Code: CSA0404

GitHub Repository:
https://github.com/ashvin280208/CSA-0404-Operating-systems.git

Total Marks: 50

Table of Contents

Table of Contents 2

1. Introduction, Method and Environment 6

1.1 Purpose of this report 6

1.2 How the scenario was interpreted 6

1.3 Evaluation method used throughout 7

1.4 Experimental environment 7

1.5 GitHub repository layout and README 8

TASK 1 – Evaluate the Requirements of Each Computing Environment (4 Marks) 9

1.1 Requirement analysis of every environment 9

1.2 Ranked requirement table 10

1.3 Which requirements conflict? 11

1.4 Which requirements are critical to patient safety? 12

1.5 Lessons from the history of operating systems 12

1.6 Alternative concept: one general-purpose OS on every device 13

1.7 Evidence summary for Task 1 14

TASK 2 – Judge the Type of Operating System for Each Environment (3 Marks) 15

2.1 The types of operating systems 15

2.2 Comparison against the hospital's environments 16

2.3 Symmetric versus asymmetric multiprocessing for the imaging server 16

2.4 Clustered system with fail-over for the database server 17

2.5 Alternative concept: one large multiprocessor instead of a cluster 17

2.6 Evidence summary for Task 2 18

TASK 3 – Evaluate Operating-System Structures (4 Marks) 19

3.1 The candidate structures 19

3.2 Effect on reliability, performance and ease of updating 20

3.3 Which structure suits the monitors and which suits the database server? 21

3.4 How some real systems are structured 21

3.5 Alternative concept: monolithic kernel instead of microkernel on the monitors 22

3.6 Evidence summary for Task 3 23

TASK 4 – Evaluate the Services an Operating System Must Provide (4 Marks) 23

4.1 The services 23

4.2 Service table for each environment 24

4.3 Which services can be removed from the bedside monitor? 25

4.4 Services essential for the database server 25

4.5 How the OS protects one patient's records from another user's programs 25

4.6 Alternative concept: tablets with a full desktop OS and every service enabled 26

4.7 Evidence summary for Task 4 27

TASK 5 – Examine System Calls (4 Marks) 27

5.1 Categories of system calls 27

5.2 What happens when a program calls read() 28

5.3 How parameters are passed to the kernel 29

5.4 Evidence: strace output from a real program 29

5.5 Alternative concept: fopen/fread/fwrite instead of open/read/write 32

5.6 Evidence summary for Task 5 33

TASK 6 – Evaluate the Cost of System Calls (4 Marks) 34

6.1 Why a system call costs much more than a function call 34

6.2 The program: copying a file with read() and write() 34

6.3 Measurements for a larger file 36

6.4 Alternative concept: memory-mapped files (mmap) 39

6.5 Evidence summary for Task 6 40

TASK 7 – Evaluate Process Control System Calls (4 Marks) 40

7.1 What fork(), exec(), wait() and exit() do 40

7.2 How a shell runs a command and obtains the exit status 41

7.3 The completed program 41

7.4 Why stdout is flushed before each fork() 42

7.5 Zombie and orphan processes 43

7.6 Why fork() followed by exec() and not one create-process call 44

7.7 Alternative concept: posix_spawn() or threads instead of fork + exec 45

7.8 Evidence summary for Task 7 46

TASK 8 – Evaluate CPU States and Protection (3 Marks) 46

8.1 User mode, kernel mode, privileged instructions and the mode bit 46

8.2 Interrupts, traps and the timer interrupt 46

8.3 What would happen to the monitors if any program could disable interrupts 47

8.4 How the hardware stops one program from reading another program's memory 47

8.5 Experiment: polling versus interrupt-driven detection 48

8.6 Alternative concept: polling instead of interrupts 49

8.7 Evidence summary for Task 8 50

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems (4 Marks) 51

9.1 The levels of the memory hierarchy 51

9.2 Where should MRI images, patient records and monitor readings be kept? 53

9.3 Programmed I/O, interrupt-driven I/O and DMA 54

9.4 The effect of caching on the speed of record look-up 54

9.5 Alternative concept: interrupt-driven I/O (one interrupt per word) instead of DMA . . . 55

9.6 Evidence summary for Task 9 55

TASK 10 – Measure and Compare (4 Marks) 56

10.1 Measurements planned 56

10.2 Number of system calls for different buffer sizes 56

10.3 Time in user mode and in kernel mode 56

10.4 Time to create 1,000 processes and 1,000 threads 57

10.5 How the measurements support or contradict the earlier judgements 59

10.5 a Memory used by processes and threads 60

10.6 Alternative concept: threads instead of processes for the measurements 60

10.7 Evidence summary for Task 10 61

TASK 11 – Recommend and Defend the Operating-System Plan (4 Marks) 61

11.1 The recommended plan 61

11.2 Why this plan satisfies the clinical concerns 62

11.3 Limitations and risks of the plan 63

11.4 Does the plan remain suitable for a group of three hospitals? 63

11.5 Alternative concept: virtual machines on two large servers 64
11.6 Evidence summary for Task 11 64
TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks) 65
A. Impact Analysis (2 Marks) 65
B. Evaluation of the Original Design (2 Marks) 66
C. Adaptation or Redesign (2 Marks) 67
D. Evidence-Based Defence (2 Marks) 69
13. Conclusion and Summary of Evidence 70
Programs and files produced 71

1. Introduction, Method and Environment

1.1 Purpose of this report

This report answers Assignment 1 of the course CSA04 – Operating Systems (Course Outcome CO1). The assignment asks the student to evaluate and justify the operating-system design of a smart hospital. Every task is assessed at Bloom's level K5 (Evaluate), so each answer in this report follows the same pattern: the requirement is stated, the candidate designs are compared against measurable criteria, a decision is made, and the decision is defended with evidence. The evidence is of three kinds: reasoning from operating-system principles, calculations in Big-O notation, and measurements taken by running C programs on a Linux machine. All programs in this report were compiled and executed, and the outputs printed here are the real outputs of those runs.

The report is organised in the same order as the assignment. Tasks 1 to 5 evaluate the requirements, the types, the structures, the services and the system-call interface of the operating system. Tasks 6 to 10 look at the cost of the interface through programs and measurements: the system-call cost of copying a file, process control with fork, exec and wait, CPU modes and interrupts, the memory hierarchy, and a comparison of processes and threads. Task 11 combines everything into one recommended plan. Task 12 is the Day-4 design challenge, in which home monitoring and a cloud records service are added and the plan is re-evaluated.

1.2 How the scenario was interpreted

The hospital has six kinds of computing environment. They are listed in Table 1.1 with the key property that dominates the choice of operating system for each of them.

Environment	Quantity (hospital stage)	Dominant property	First consequence for the OS
Bedside patient monitor	500	Alarm within 10 ms of an abnormal reading	Hard real-time scheduling, tiny kernel
Nurse handheld tablet	120	Battery must last a full shift	Handheld OS with aggressive power management
Doctor desktop workstation	80	Interactive response, many applications	General-purpose time-sharing desktop OS
Central database server	2-3 nodes	Must never stop, even during updates	Clustering with fail-over, journaling file system
Imaging (MRI) server	1-2 machines	High throughput on many processors	Symmetric multiprocessing, DMA, large memory

Environment	Quantity (hospital stage)	Dominant property	First consequence for the OS
Partner-hospital link	2 sites, later 3 hospitals	Secure sharing of records	Distributed system, authentication, encryption

The scenario also states that the system will grow: one ward (40 monitors, 10 tablets), then one hospital (500 monitors, 120 tablets, 80 workstations), then a group of three hospitals. Growth is important because it changes the cost of administration. A design that needs a human to log in to each device is acceptable for 50 devices but not for 2,000 devices, so the plan must favour central management, automated updates and a small number of operating-system families.

1.3 Evaluation method used throughout

Each design alternative is scored against criteria taken from the scenario. The criteria are called R1 to R7 in the rest of the report.

- R1 – Response time: how long from an event (a reading, a key press) to the system's reaction, and in the worst case, not only the average.
- R2 – Reliability and availability: how long the system runs without failing and how quickly it recovers.
- R3 – Throughput: how much work is completed per unit time.
- R4 – Resource use: memory, storage and CPU consumed by the OS itself.
- R5 – Power: energy use, which is decisive for battery devices.
- R6 – Security and privacy: protection of patient records, authentication, auditability.
- R7 – Administration effort: the cost of updating and monitoring the devices as the number grows.

Where a requirement affects patient safety it is marked *critical*. The word "critical" is used in a strict sense: a failure of this requirement can directly harm a patient. Alarm latency and database availability are critical. Battery life is not directly critical, but a flat battery on a tablet during a medication round has safety consequences, so it is treated as *important*.

1.4 Experimental environment

The programs were executed on a Linux virtual machine. The details are given here so that the measurements can be reproduced and interpreted fairly.

Item	Value
Operating system	Ubuntu 24.04 LTS (Linux kernel 6.18, virtualised)
Compiler	gcc 13.3.0

Item	Value
Compile command	gcc -std=c11 -Wall -Wextra -pthread -O2 file.c -o file
CPU cores available	1 virtual CPU (important for the thread and multiprocessor discussions)
Timing sources	clock_gettime(CLOCK_MONOTONIC), getrusage(), the bash time keyword

Because the machine has a single virtual CPU and runs inside a sandbox, absolute times are larger and noisier than on a physical workstation, and the strace utility was not installed. The *ratios* between methods (for example buffer 1 byte versus buffer 4096 bytes) are still valid, and they are the basis of the conclusions. Where the assignment asks for strace output, this report explains exactly what the output would contain, shows the equivalent counts that the programs print, and gives the command that the student must run on the personal machine for the repository screenshots.

1.5 GitHub repository layout and README

The assignment requires a repository called CSA04-<RegisterNumber>-Assignment1 with one folder per task, a README.md, and at least one commit per task. The layout used for this work is shown below.

```
CSA04-<RegisterNumber>-Assignment1/
  README.md
  Task01/ requirement_table.md
  Task02/ os_types.md
  Task03/ structures.md
  Task04/ services.md
  Task05/  strace_read.txt  (output of: strace -c ./t06_copy)
  Task06/ t06_copy.c  t06_bench.c  outputs/
  Task07/ t07_shell.c  t07_noflush.c  t07_zombie.c  outputs/
  Task08/ t08_polling.c  outputs/
  Task09/  t09_latency.c  outputs/
  Task10/ t10_create.c  outputs/  time_outputs.txt
  Task11/ plan.md
  Task12/  redesign.md
```

The README.md must contain the name, the register number, the compiler and version (gcc 13.3.0), the Linux distribution and version, the exact compile command, the run steps of each program and a one-line result for each task. A suitable commit sequence is: "Task 1: requirement table", "Task 2: OS types", and so on, one commit per task, so that the history shows progressive work.

> The student's name, register number and repository link are shown as placeholders on the cover page. They must be filled in before submission. The programs and their outputs should be re-run by the student on their own machine; the viva may ask to

explain or modify any of them.

TASK 1 – Evaluate the Requirements of Each Computing Environment (4 Marks)

1.1 Requirement analysis of every environment

Bedside patient monitors

A bedside monitor reads sensors such as heart rate, blood oxygen, blood pressure and ECG at rates between 1 Hz and 500 Hz. The scenario says an alarm must be raised within 10 ms of an abnormal reading. This is a *deadline*, and a deadline is different from "fast". A system that responds in 1 ms on average but in 80 ms once in a thousand times has failed the requirement, because the late alarm is exactly the one that may matter. The relevant property is therefore the *worst-case* latency, which depends on how long interrupts can be disabled, how long the kernel can be non-preemptible, and whether lower-priority work can hold a resource that the alarm task needs (priority inversion).

A monitor serves one patient and runs a handful of fixed tasks (acquire, filter, display, alarm, network report). It needs no multi-user support, no large file system and no general-purpose user interface. Its memory is small (often 64 MB to 1 GB of RAM in a modern monitor, or even kilobytes in a simple sensor node). It is mains powered, but the battery backup matters during transport. Security is important because the device is on the hospital network and measurement data are personal data, but the device has a small attack surface because it runs only signed firmware.

Nurse handheld tablets

A tablet is carried for a whole shift and is used for medication scanning, charting and alerts. The dominant requirement is energy: the OS must suspend the CPU and screen aggressively, wake only for events, and batch network activity. Interactive response (touch to reaction within 100 ms) is needed, but that is a *soft* requirement; occasional delays are annoying but not dangerous. Security is crucial because the device can be lost or stolen: it needs full-disk encryption, a screen lock, remote wipe and application sandboxing.

Doctor desktop workstations

Workstations are plugged in and used by one person at a time for many applications: electronic health record (EHR) client, image viewer, e-mail, web. The OS must provide a rich graphical interface, preemptive time-sharing and good support for peripherals. Throughput is moderate and response is soft real time. The administration concern is large because there are 80 machines (and later more), so central policy, automatic patching and

user authentication against the hospital directory are important.

Central patient-records database server

The database stores every patient's history. The scenario says it must never stop, even during updates. The requirement is *availability* (for example 99.99 %, which allows about 52 minutes of downtime per year), together with durability (no committed record is ever lost) and consistency. Throughput is high because every ward writes and reads continuously. The OS must therefore support a journaling file system, hot-swappable disks, rolling updates, many processors, large memory for caching, and a clustered mode in which a second node takes over within seconds. Security is the highest of all environments because the data are confidential.

Imaging (MRI) server

An MRI scan can be hundreds of megabytes and reconstruction is computation-heavy. The server needs *throughput*: many processors working on one problem, a very large memory, and fast bulk I/O. Response time is measured in seconds and minutes, not milliseconds. Fault tolerance matters less than for the database because a scan can usually be reprocessed, though the raw data must be stored safely.

Partner-hospital link

The link shares records with two partner hospitals. The OS must provide strong networking, a trustworthy authentication mechanism, encryption in transit and an audit trail. It must tolerate slow and unreliable wide-area links, so communication must be asynchronous and retried. This is the environment where distributed-system services (naming, authentication, replication) matter most.

1.2 Ranked requirement table

Table 1.2 ranks the requirements of each environment from 1 (most important) to 7 (least important). The justification column explains the ranking. The ranking was obtained by asking, for each requirement, "what is the consequence of failure?" and "how likely is conflict with another requirement?".

Environment	R1 Response	R2 Reliability	R3 Throughput	R4 Footprint	R5 Power	R6 Security	R7 Admin	Justification of the ranking
Bedside monitor	1	2	6	3	5	4	7	A late alarm harms the patient (R1); a crash removes monitoring (R2); small footprint enables determinism (R4); security is needed but attack surface is small.
Nurse tablet	4	3	7	5	1	2	6	The device must last a shift (R5); loss or theft exposes data (R6); a crash is recoverable.
Doctor workstation	2	4	5	7	6	3	1	Interactive use (R2 response); many devices, so central administration dominates (R7).
Database server	3	1	2	7	6	1	5	Never stopping is the requirement (R2); throughput of many ward users (R3); data privacy (R6 tied for first).
Imaging server	6	3	1	5	7	4	2	Computation volume is the purpose (R3); hardware is shared by few users so admin is simple.
Partner link	4	2	5	6	7	1	3	Confidential data cross an untrusted network (R6); retry and recovery matter (R2).

Ranking ties are allowed where two requirements are inseparable, as for the database server where confidentiality and availability both have the worst consequences.

1.3 Which requirements conflict?

Operating-system design is a negotiation between requirements that pull in opposite directions. The most important conflicts in the hospital are listed below.

1. **Fast response versus many users on one machine.** Time-sharing gives every user a fair share of the CPU, which is the opposite of guaranteeing that one task meets a deadline. If one machine ran the alarm task and 80 interactive users, a burst of user activity could delay the alarm. Real-time scheduling solves this by giving the alarm task the highest fixed priority, which is only acceptable when the number of high-priority tasks is small and their total demand is known in advance.
2. **Throughput versus latency.** Batching improves throughput (fewer context switches, longer time slices, larger disk requests) but increases the delay before an individual request is served. The imaging server wants batching; the monitor cannot afford it.
3. **Features versus footprint and security.** Every additional service (file system, GUI, network stack, USB) adds code, memory and attack surface. A monitor with a USB stack that it does not need is both slower to boot and easier to attack.
4. **Power versus responsiveness.** Deep sleep saves battery but a device in deep sleep takes milliseconds to wake. A tablet may accept this; a monitor cannot.
5. **Availability versus consistency and cost.** A fail-over cluster improves availability but requires replication, and replication complicates consistency and raises cost.
6. **Security versus usability.** Strict authentication slows a nurse who needs a record immediately. The design must give emergency access that is logged rather than blocked.

1.4 Which requirements are critical to patient safety?

Patient-safety critical requirements are the ones where the failure itself can injure a patient. They are: (a) the 10 ms alarm deadline on the monitors, (b) the availability and integrity of the patient database, (c) the correct identification and authorisation of users (a wrong record shown to a clinician is a safety hazard), and (d) integrity of the alarm path from the monitor to the nurse's tablet, because an alarm that is raised but never delivered is as dangerous as one that is never raised. The IT team's single-OS proposal threatens (a) and (b) directly, as the clinical staff suspected.

1.5 Lessons from the history of operating systems

Historical stage	Key idea	Where it applies in the hospital	Lesson
Batch systems (1950s-60s)	Jobs are run one after another without interaction, to keep an expensive CPU busy	MRI reconstruction queue, nightly backups and billing reports	Throughput and CPU utilisation matter more than response; a job queue with priorities is appropriate
Multiprogramming and time-sharing (1960s-70s)	Many jobs share the CPU; the CPU switches on a timer; interactive response	Database server serving many wards; doctor workstations	Fair sharing gives good average response but no guarantee for any single task
Personal computers (1980s-)	One user, one machine, graphical interface, convenience over utilisation	Doctor workstations	Security and multi-user protection were once ignored and later had to be added; the hospital must not repeat this
Distributed and clustered systems (1990s-)	Many machines cooperate through a network	Database cluster, partner-hospital link	Failure of one node should not stop the service; networks introduce partial failure
Real-time systems	Correctness depends on meeting the deadline	Bedside monitors	Predictability is more important than speed
Mobile and handheld (2000s-)	Battery, touch, always connected, sandboxed apps	Nurse tablets	Power management and application isolation are first-class OS functions

The history shows that each new environment produced a new kind of operating system because the older one optimised the wrong quantity. Batch systems optimised utilisation, time-sharing optimised average response, and real-time systems optimise worst-case response. A single general-purpose OS that is tuned for the average cannot be tuned for the worst case at the same time.

1.6 Alternative concept: one general-purpose OS on every device

What the solution would be. The IT team proposes a single general-purpose operating system (for example a standard Linux or Windows distribution) on all devices.

Worst-case alarm response. In a general-purpose kernel the scheduler aims at fairness and throughput. A task that becomes runnable must wait for the running task to be preempted, for any kernel code that runs with preemption disabled, and for interrupts that are being serviced. There is no upper bound that can be proved. Measurements published for stock

Linux under load show worst-case wake-up latencies from a few milliseconds to more than 100 ms, which violates the 10 ms requirement. A real-time OS (or Linux with the PREEMPT_RT patch set and CPU isolation) gives a bounded worst case, usually in the range of tens of microseconds, which satisfies the requirement with a wide margin. In Big-O terms the issue is not the average cost but the existence of a bound: the scheduling latency of the general-purpose OS is *unbounded in theory* (it depends on the load), while the real-time kernel has an O(1) scheduler and bounded critical sections, giving a bound that is independent of load.

Memory. The memory per device grows because the general-purpose OS includes everything. The table below uses typical figures.

Device	Real-time/embedded OS footprint	General-purpose OS footprint	Consequence
Bedside monitor	0.1 - 5 MB flash/RAM for kernel and services	0.5 - 2 GB of disk and 300 - 800 MB of RAM at idle	Larger and more expensive hardware, slower boot (seconds to minutes instead of under 1 s)
Tablet	Android-class: about 1 GB	Desktop OS: 2 - 4 GB RAM and 20 GB+ disk	Faster battery drain, because more background services keep the CPU awake
Workstation	Not applicable	Desktop OS	Same in both designs

Administration. One OS family reduces the *number of skills* needed by the IT staff, which is a real advantage, and a single patching process. However, the patching process becomes the hazard: a single update applied to 500 monitors and the database simultaneously can stop everything at once. With different OS types, the monitors are updated on their own cycle (rarely, with validation), the database nodes are updated one at a time, and the tablets through a mobile-device-management system. The administration effort is O(d) in the number of devices d for per-device manual work in both designs, but the single-OS design has fewer *kinds* of procedure and a larger *blast radius* for each mistake.

Judgement. The alternative is rejected for the monitors and the database server, where its worst-case behaviour breaks critical requirements. It is acceptable (and in fact preferable) for the workstations and for the imaging server, where a mainstream Linux or Windows is the right choice. The conclusion for Task 1 is therefore a *mixed* design that uses at most three OS families: an RTOS on the monitors, a handheld OS on tablets, and Linux (with appropriate configuration) on the servers and gateways, with a desktop OS on the workstations.

1.7 Evidence summary for Task 1

The ranked requirement table (Table 1.2), the conflicts, the patient-safety requirements and the quantitative comparison with the single-OS alternative together justify the decision to use different operating-system types in the different environments. Section 10 later checks this judgement with measurements.

TASK 2 – Judge the Type of Operating System for Each Environment (3 Marks)

2.1 The types of operating systems

Mainframe systems. Mainframe operating systems (such as z/OS) are designed for very large numbers of concurrent jobs and transactions, extreme reliability, hot-swappable hardware and strong I/O capacity. They combine batch, transaction processing and time-sharing. Their advantage is availability measured in years; their disadvantages are cost and the need for specialised staff.

Desktop systems. A desktop OS (Windows, macOS, a Linux desktop) serves one interactive user. It optimises responsiveness, supports a graphical interface and a wide range of peripherals, and uses time-sharing and virtual memory. Protection between users exists but the focus is on convenience.

Multiprocessor (parallel) systems. These have more than one CPU sharing memory and the bus. They increase throughput and reliability (graceful degradation). Two organisations exist: *asymmetric multiprocessing* (AMP), in which one master processor runs the OS and assigns work to slaves, and *symmetric multiprocessing* (SMP), in which every processor runs the same OS and can execute kernel code.

Distributed systems. A distributed system is a collection of independent computers that appear to users as one system. The computers communicate by messages over a network. Benefits are resource sharing, computation speed-up, reliability and communication. The new problems are partial failure, network delay and security.

Clustered systems. Clusters are groups of computers connected by a fast network and managed together to provide one service. In *asymmetric clustering* one node is on hot standby and monitors the active node; in *symmetric clustering* all nodes run applications and monitor each other. Clustering gives high availability through fail-over.

Real-time systems. In a *hard* real-time system a missed deadline is a failure (the result is useless or harmful). In a *soft* real-time system a missed deadline degrades quality but is tolerable (video playback). A real-time OS offers priority-based preemptive scheduling, bounded interrupt latency, bounded kernel critical sections and deterministic memory

allocation.

Handheld systems. These are small battery-powered devices (phones, tablets, PDAs) with limited memory, small screens and slow processors relative to desktops. Their operating systems emphasise power management, touch interface, wireless networking and sandboxed applications.

2.2 Comparison against the hospital's environments

Environment	Best-fitting type	Reason	Rejected alternatives and reasons
Bedside monitor	Hard real-time (embedded RTOS such as QNX, VxWorks, FreeRTOS or Zephyr on a certified device)	The 10 ms deadline must be met every time; small deterministic kernel	Desktop OS: unbounded latency; handheld OS: soft real time only and heavy; mainframe: irrelevant scale
Nurse tablet	Handheld (Android-class or iPadOS) with mobile-device management	Battery, touch, wireless, application sandbox	Desktop OS: heavy power use and large footprint; RTOS: no application ecosystem
Doctor workstation	Desktop (Windows or Linux desktop) with central management	Interactive use, the EHR client and imaging viewers run there	Handheld OS: screen and input unsuited; RTOS: no desktop software
Database server	Clustered system on a server OS (Linux), possibly with a mainframe-class platform for very large sites	Fail-over within seconds, journaling, many users	Single machine: single point of failure; desktop OS: not designed for 24x7 service
Imaging server	Symmetric multiprocessor (Linux SMP) with DMA and large memory	One OS image shares all cores and memory; compute-bound parallel work	AMP: master is a bottleneck; cluster: overhead of distributing 500 MB images over a network
Partner link	Distributed system services on server OS (authenticated, encrypted, asynchronous message exchange)	Independent organisations, unreliable WAN, privacy	Tight cluster: sites do not trust each other and cannot share a fast private network

2.3 Symmetric versus asymmetric multiprocessing for the imaging server

In asymmetric multiprocessing the master processor executes the operating system and distributes the user work. Its advantage is simplicity: kernel data structures need no locking because only one CPU touches them. Its disadvantage is that the master becomes the bottleneck as the number of processors increases, and a failure of the master stops the

whole machine. In symmetric multiprocessing all processors are peers: each runs its own scheduler on a shared run queue or per-CPU run queues, and any processor can run kernel code. Kernel data structures must be protected by locks, which complicates the kernel and adds overhead, but throughput scales much better.

For the imaging server, reconstruction of MRI images is compute-bound, parallel by slice, and memory-bandwidth hungry. SMP is the right choice because every processor can run a worker and can handle its own interrupts and DMA completions. Linux is an SMP kernel with per-CPU run queues and NUMA awareness. The expected speed-up follows Amdahl's law: with a serial fraction s, the speed-up on p processors is 1 / (s + (1 - s) / p). For s = 0.05 and p = 32 the speed-up is 12.5, and for p = 64 it is 15.4, which shows that the serial fraction (reading the file, merging results) is what the designers must reduce.

The same machine can be shared with other users, so the number of processors must be matched to the memory bandwidth. In SMP systems with non-uniform memory access (NUMA), the memory of the local node is faster than remote memory, so the OS should place a worker's pages on the node where it runs.

2.4 Clustered system with fail-over for the database server

In the proposed cluster, two or three database nodes share a replicated or shared-storage volume and exchange *heartbeat* messages every second. If the passive node misses a threshold of heartbeats (for example three), it concludes that the active node has failed, *fences* it (powers it off or cuts its storage access so that two nodes never write at the same time), mounts or promotes the replicated storage, starts the database service, replays the journal and takes over the service IP address. A typical fail-over time is 5 to 30 seconds. Because clients reconnect automatically, the staff see a brief pause rather than a long outage.

The cluster also allows *rolling updates*: the passive node is updated and tested first, the service is switched to it, and then the former active node is updated. This answers the clinical concern that the database would stop during updates. The price is that every write must be replicated to the standby (synchronous replication adds a network round trip to each commit, typically 0.2 to 1 ms on a hospital LAN).

2.5 Alternative concept: one large multiprocessor instead of a cluster

What the solution would be. The database runs on one very large SMP machine (for example 128 cores and 4 TB of memory) with redundant power supplies and hot-swap disks, but only one operating-system image.

Availability. Availability is A = MTBF / (MTBF + MTTR). The single machine is a *single point of failure*: a kernel panic, a motherboard fault or a firmware update stops the service until repair. If the mean time to repair is 4 hours and the MTBF is 5 years (43,800 hours), the availability is 43,800 / 43,804 = 99.991 %, which allows about 48 minutes of downtime per year, but each failure is a 4-hour outage during which no patient record can be read, which is unacceptable in a hospital even if the yearly average looks good. The cluster has a repair time (fail-over) of about 15 seconds, and two independent nodes fail together with probability about (1 - A)^2 for short windows, so availability of five nines (99.999 %) is realistic.

Cost. Large SMP machines cost super-linearly with the number of processors, because the coherent interconnect and the memory system are expensive. A cluster of commodity two-socket servers costs roughly linearly. For the same total compute, the large machine can cost three to ten times more, and it still does not remove the single point of failure.

Throughput growth (time). On the single machine the throughput grows with the number of processors p according to Amdahl's law, T(p) = T(1) (s + (1 - s)/p). Locks inside the database and the kernel, and cache-coherence traffic, make s larger as p grows, so the speed-up flattens (for example 32 cores give about 18 times the throughput of one core). In the cluster the read capacity grows almost linearly with the number of nodes (reads can be served from replicas), but writes do not scale because every write is replicated; so for a hospital workload, which is read-heavy (perhaps 90 % reads), the cluster scales better.

Memory (space). The large machine allows a very large cache (the whole working set in RAM), which is a real advantage for look-up speed. In the cluster each node has its own cache, so the aggregate memory is large but the cached data is duplicated on each node, giving less effective capacity (space O(k*m) for k nodes with m memory each, but the useful distinct data is closer to O(m)).

Judgement. The cluster with fail-over is chosen. The large multiprocessor is a good choice for the *imaging* server (where a single big memory helps and downtime is tolerable) but not for the patient database, where the single point of failure conflicts with the explicit requirement that it must never stop.

2.6 Evidence summary for Task 2

The justified choices are: hard real-time OS for the monitors; handheld OS for tablets; desktop OS for workstations; clustered Linux with fail-over for the database; SMP Linux for imaging; distributed secure services for the partner link. The rejected alternatives and the reasons are given in Table 2.1 above.

TASK 3 – Evaluate Operating-System Structures (4 Marks)

3.1 The candidate structures

Figure 3.1 Operating-system structures

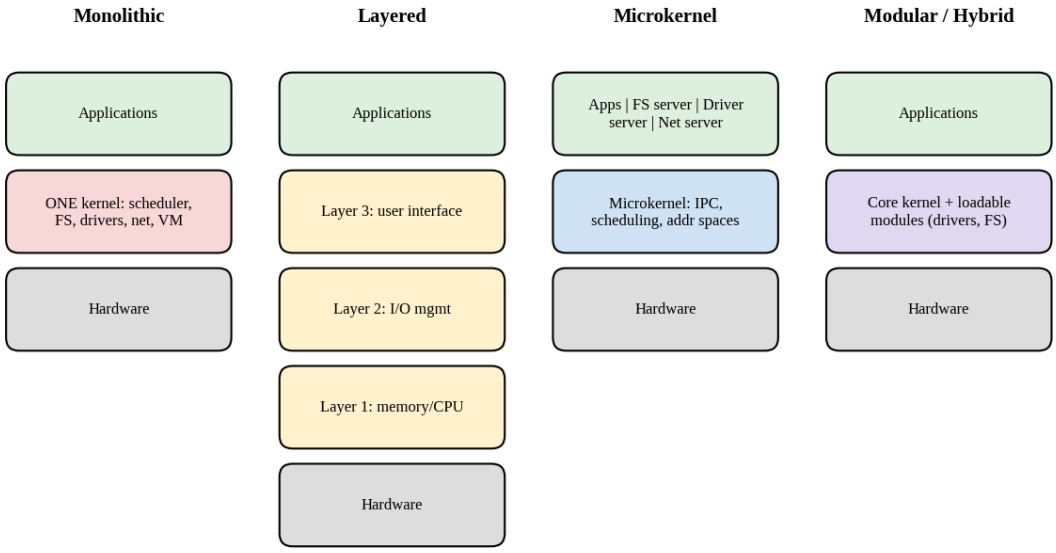


Figure 3.1 Simple, layered, microkernel and modular/hybrid structures

Simple (monolithic) structure. In a monolithic kernel all operating-system services (process scheduling, memory management, file systems, device drivers, networking) run in one address space in kernel mode and call each other as ordinary function calls. Traditional UNIX and Linux are essentially monolithic. The performance is high because a service request costs one system call and internal calls cost almost nothing. The weakness is that a bug in any component, for example a faulty driver, can overwrite any kernel data and crash the whole system; and the kernel is large, so it is hard to verify.

Layered structure. The OS is divided into layers, with layer 0 the hardware and layer N the user interface. Each layer uses only the services of the layers below it. This gives simplicity of debugging and verification (each layer can be tested on top of the lower, already tested layers), but every request crosses several layers, each of which adds parameter passing and checks. It is also difficult to decide which function belongs to which layer (for example, the backing store driver must be below memory management but memory management must be above the scheduler that blocks processes). Pure layering is therefore rare; the idea is used in a relaxed form (the network protocol stack is a good example).

Microkernel structure. Only the minimum stays in the kernel: address-space management, thread scheduling and inter-process communication (IPC). File systems,

device drivers, the network stack and the windowing system run as user-level *servers*. Applications request a service by sending a message to a server through the kernel. A faulty driver crashes only its own process, which can be restarted, and the kernel is small (tens of thousands of lines, which makes formal verification possible, as in seL4). The cost is performance: a service request needs at least two message transfers and four mode switches (client to kernel to server and back) instead of one trap.

Modular structure (loadable kernel modules). The kernel has a core with well-defined interfaces, and components (drivers, file systems, schedulers) can be loaded into and removed from the kernel at run time. This is how modern Linux, Solaris and many other kernels are organised. It combines the speed of a monolithic kernel with flexibility: a new device driver is added with `modprobe` and the system need not be rebuilt or rebooted. A loaded module still runs in kernel mode, so it can still crash the kernel.

Hybrid structure. Real systems mix the approaches to obtain a compromise. Windows NT-family kernels keep many services in kernel mode for speed but keep a layered organisation and a user-mode subsystem model; macOS and iOS combine a Mach microkernel with a BSD monolithic component and loadable drivers (I/O Kit); Android uses a Linux kernel (monolithic with modules) with a user-space framework (ART, system services) on top.

3.2 Effect on reliability, performance and ease of updating

Structure	Reliability	Performance	Ease of updating	Typical examples
Monolithic	Low isolation: one faulty component can crash everything	Highest: a system call is one trap; internal calls are direct	Whole kernel replaced and rebooted	Early UNIX, MS-DOS, FreeRTOS in its common configuration (single address space)
Layered	Good fault localisation if layers are checked	Lower: crossing layers costs time	Change inside a layer affects only that layer if the interface is kept	THE system (Dijkstra), network protocol stacks
Microkernel	Best isolation; servers restarted on fault	Lower: message passing and extra mode switches	Servers can be replaced at run time without rebooting	QNX, MINIX 3, seL4, L4, Mach (original)
Modular (LKM)	Moderate: modules share the kernel address space	Near monolithic	Modules loaded and unloaded without rebooting (but core changes need a reboot)	Linux, Solaris, FreeBSD

Structure	Reliability	Performance	Ease of updating	Typical examples
Hybrid	Depends on mix; drivers in kernel reduce isolation	Good	Moderate	Windows NT/10/11, macOS/iOS (XNU), Android

3.3 Which structure suits the monitors and which suits the database server?

Patient monitor – microkernel real-time OS. The monitor must be highly reliable (a crash of the display driver must not stop the alarm) and predictable. A microkernel RTOS such as QNX isolates the display driver, the network stack and the file system in separate processes, so that the alarm task keeps running if any of them fails. The kernel is small and has short, bounded non-preemptible sections, which gives a small and provable worst-case latency. The extra message-passing cost is not a problem because the monitor's data rates are low (a few kilobytes per second) and the 10 ms budget is large compared with an IPC of a few microseconds. Where hardware is extremely limited (a sensor node with 64 KB of RAM) a monolithic RTOS such as FreeRTOS, with all tasks in one address space and an MPU (memory-protection unit) to protect critical tasks, is the practical alternative.

Database server – monolithic/modular kernel (Linux). The database server needs the highest possible I/O and network performance for a large number of requests; each avoidable mode switch costs throughput. The modular monolithic Linux kernel gives direct, fast paths from the database through the VFS and block layer to the NVMe driver, and its drivers and file systems are tried and tested. Reliability is achieved at a higher level, by the cluster, rather than by the kernel structure: if a node's kernel panics, the other node takes over.

3.4 How some real systems are structured

System	Structure	Notes
Linux	Monolithic kernel with loadable modules	Everything in kernel mode; modules for drivers and file systems; user space has glibc, systemd and applications; supports PREEMPT_RT for real time
Windows	Hybrid (NT executive in kernel mode, layered, with user-mode subsystems)	HAL isolates hardware; many drivers in kernel mode; some (UMDF) in user mode; Windows is not hard real time

System	Structure	Notes
Android	Linux kernel + hardware abstraction layer + native libraries + Android Runtime + application framework + applications	Layered stack on top of a modular monolithic kernel; each app runs in its own sandbox (Linux user ID and SELinux)
FreeRTOS	Small monolithic real-time kernel (scheduler, queues, semaphores, timers) linked with the application into one image	Footprint of 6-12 KB; optional MPU support; no processes by default, only tasks
QNX	True microkernel with message passing; drivers, file systems and network as user-level processes	Used in medical devices and automotive; certified to safety standards; bounded latency

3.5 Alternative concept: monolithic kernel instead of microkernel on the monitors

What the solution would be. The monitor firmware uses a monolithic kernel (for example Linux with a driver for the ECG front end and the display driver compiled in, or FreeRTOS with all drivers as ordinary functions).

Time cost of a service request. In a monolithic kernel a request such as "read the sensor sample" is one system call: user to kernel, run the driver code, return: cost *c_trap* of about 0.1 to 1 µs on a modern CPU. In a microkernel the same request is: client traps to the kernel (send), the kernel switches to the driver server, the server handles the request and replies, the kernel switches back: two IPC operations and about four mode switches, costing 1 to 10 µs on a good microkernel (L4-family kernels achieve roughly 0.3 µs one way for short messages, QNX a few µs). In big-O terms both are O(1) per request, but with a constant 3 to 10 times larger for the microkernel. If a monitor performs 1,000 service requests per second, the overhead is at most 10 ms of CPU time per second, which is 1 %, a cost that is acceptable. The microkernel latency is also *bounded and short*, so 10 µs is much less than the 10 ms alarm deadline.

Memory taken by the kernel. A monolithic Linux kernel image is 5 to 30 MB with its modules; a minimal monolithic RTOS kernel is 6 to 50 KB. A microkernel is tens to a few hundred KB, but each server is a separate process with its own stack and address space, which adds some tens of KB per server. The total for a full microkernel system (kernel + servers) may be similar to a small monolithic kernel, but the *trusted computing base* (the code that, if wrong, can crash everything) is much smaller: perhaps 10,000 lines against more than 20 million lines for Linux.

Effect of a faulty driver. In the monolithic design a wild pointer in the display driver can overwrite the alarm task's data or the scheduler's queue, and the kernel panics: *all* monitoring stops until reboot (several seconds to a minute for a Linux boot). In the microkernel the display server dies, the kernel notifies a supervisor, the server is restarted in tens of milliseconds, and the alarm task, which is a separate process, never notices. For a safety-critical device, fault containment is worth much more than a few microseconds per call, so the microkernel (or an RTOS with memory protection between tasks) is chosen for the monitors.

Summary of the comparison.

Criterion	Monolithic	Microkernel
Time per service request	1 trap, O(1), about 0.3 µs	2 IPC + 4 mode switches, O(1), about 2-5 µs
Kernel memory	Large (MB) or small RTOS (KB) but one fault domain	Small kernel + servers, many fault domains
Effect of faulty driver	System crash	Server restart
Worst-case latency analysis	Hard: long kernel paths	Easier: short kernel, bounded IPC

3.6 Evidence summary for Task 3

Two defended choices have been made: a microkernel (or memory-protected) real-time OS for the monitors, because fault isolation and bounded latency are worth the small IPC cost, and a modular monolithic Linux kernel for the database server, because maximum I/O performance is needed and availability is provided by clustering above the kernel.

TASK 4 – Evaluate the Services an Operating System Must Provide (4 Marks)

4.1 The services

An operating system provides two kinds of services: those that help the user and those that ensure efficient operation of the system itself.

Service	What it does	Why it exists
User interface	Command-line interpreter, graphical interface or touch screen	Lets people control the machine
Program execution	Loads a program into memory, runs it, ends it normally or with an error	Without it nothing runs

Service	What it does	Why it exists
I/O operations	Gives programs access to devices and files without direct hardware access	Protects the devices and hides their details
File-system manipulation	Create, delete, read, write, search files and directories; manage permissions	Persistent storage of records
Communication	Message passing and shared memory between processes on one machine or across a network	Cooperating programs, distributed system
Error detection	Detects CPU, memory, I/O and program errors and takes the right action	Reliability
Resource allocation	Shares CPU, memory, disk and devices among competing jobs	Fairness and efficiency
Accounting	Records who uses how much of which resource	Billing, planning, troubleshooting
Protection and security	Controls access to resources; authenticates users; defends against external attacks	Confidentiality and integrity of data

4.2 Service table for each environment

The table uses E for essential, O for optional and X for removed.

Service	Monitor (RTOS)	Tablet (handheld)	Workstation (desktop)	Database server	Imaging server	Partner link
User interface	X (fixed display only)	E (touch)	E (GUI)	X (remote shell only)	X (remote shell)	X
Program execution	E (fixed task set, loaded at boot)	E (app lifecycle)	E	E	E (batch jobs)	E
I/O operations	E (sensor ADC, display, network)	E (touch, camera, barcode, radio)	E	E (NVMe, network)	E (DMA from scanner, NVMe)	E (network)
File system	X or minimal (flash log)	E (flash)	E	E (journaling)	E (large striped volume)	X
Communication	E (to gateway)	E (Wi-Fi)	E	E (replication, clients)	E	E (TLS)

Service	Monitor (RTOS)	Tablet (handheld)	Workstation (desktop)	Database server	Imaging server	Partner link
Error detection	E (watchdog, sensor fault)	E	E	E (ECC memory, disk checks)	E	E
Resource allocation	E (fixed priority)	E (power-aware)	E (time-sharing)	E (I/O fairness, cgroups)	E (CPU/NUMA)	O
Accounting	X	O	O	E (audit)	O	E (audit)
Protection and security	E (signed firmware, MPU)	E (sandbox, encryption)	E (users, ACLs)	E (strongest)	E	E (authentication, encryption)

4.3 Which services can be removed from the bedside monitor?

The monitor runs a fixed set of tasks, so it does not need a general user interface (only a fixed display driver), accounting, multi-user login, a general-purpose file system (a small flash-based log is enough), dynamic loading of programs, virtual memory with swapping (swapping is unpredictable, since a page fault to disk can take milliseconds, which would break the 10 ms deadline), a shell, a package manager or most network protocols (a single secure protocol is enough). Removing these has four benefits: (1) the kernel is smaller, so the device boots in less than a second and has less memory to protect; (2) the code that can fail is smaller, which helps certification; (3) no unpredictable services run in the background, so latency is deterministic; (4) the attack surface is smaller because services that are not present cannot be exploited.

4.4 Services essential for the database server

Essential services are: a **journaling file system** (so that a power failure never corrupts the records), **I/O operations** with fast block-layer scheduling and asynchronous I/O, **resource allocation** with control groups so that a backup cannot starve the database, **communication** for client connections and cluster replication, **error detection** (ECC memory, SMART and RAID monitoring, watchdog), **protection and security** (users, roles, encrypted storage, audit), and **accounting/audit** (who read which record, to satisfy privacy law). The graphical interface is not needed.

4.5 How the OS protects one patient's records from another user's programs

Protection is provided in several layers, each by a different mechanism.

1. **Hardware memory protection.** Every process runs in its own virtual address space, translated by the MMU. A process cannot name another process's memory; an attempt to access an address that is not mapped raises a page fault and the OS terminates the process (segmentation fault). Section 8 describes this in more detail.
2. **User and kernel mode.** Only the kernel can execute privileged instructions such as I/O or changing page tables. A program that wants to read a file must ask the kernel through a system call, and the kernel checks permissions.
3. **File permissions and access-control lists.** Each file or database object has an owner, a group and permission bits (read, write, execute for owner, group, others) or an ACL listing exactly which users or roles may read or write. A nurse's account has read access only to the records of patients on her ward.
4. **Authentication.** Users prove identity (password plus token or smartcard); the OS maps the authenticated identity to a user ID that all later checks use.
5. **Encryption.** Disk encryption protects records if a disk is stolen, and TLS protects them on the network.
6. **Auditing.** Every access to a record is logged with the user ID, time and action, so misuse can be detected afterwards.
7. **Mandatory access control.** SELinux or AppArmor confine even a compromised database process to the files it needs.

4.6 Alternative concept: tablets with a full desktop OS and every service enabled

What the solution would be. The 120 tablets would run a full desktop operating system (for example a desktop Linux or Windows) with all services: full file-system indexing, printing, multi-user login manager, desktop search, update service, remote desktop and so on.

Memory and storage (space). A desktop OS with a graphical environment uses roughly 1.5 to 3 GB of RAM at idle and 15 to 30 GB of disk; a handheld OS tuned for the hardware uses about 0.5 to 1 GB of RAM and 4 to 8 GB of storage. On a tablet with 4 GB of RAM, the desktop OS leaves much less memory for applications, causing more paging. Storage space grows with the number of services, $S = O(s)$ in the number s of enabled services. The additional storage is wasted on a device that needs none of them.

Start-up time and battery life (time). Each service started at boot lengthens the start-up. A desktop OS typically boots in 20 to 60 seconds on tablet-class hardware, while a handheld OS resumes from suspend in about a second. More significantly, background

services wake the CPU repeatedly, preventing deep sleep states, so battery life drops. A rough model: if the idle power of the handheld OS is 0.3 W and that of the desktop OS 1.2 W, a 30 Wh battery lasts 100 hours against 25 hours on standby; with screen and radio on in use (say 2 W extra), the lasts are 13 and 9.4 hours, so the desktop OS is unlikely to survive a 12-hour shift. The clinical staff's concern about batteries is therefore justified.

Judgement. A reduced handheld OS under mobile-device management (MDM) is better for the tablets. The desktop OS remains the choice for workstations where power is not limited and the services are used.

4.7 Evidence summary for Task 4

The service table (Section 4.2) shows an explicit inclusion or exclusion for every service in every environment. The monitor loses the interface, accounting, file system and virtual memory; the database keeps the journaling file system, protection and audit. Protection of one patient's record against other users is multi-layered (hardware, modes, permissions, authentication, encryption and audit).

TASK 5 – Examine System Calls (4 Marks)

5.1 Categories of system calls

A system call is the interface through which a user program requests a service from the kernel. On Linux there are more than 300 system calls, grouped into the following categories.

Category	Purpose	Examples (Linux)
Process control	Create, end, load, wait for, and control processes; allocate memory	fork, execve, exit, wait4/waitpid, kill, brk, mmap, clone
File management	Create, delete, open, close, read, write, reposition files	open/openat, read, write, close, lseek, unlink, stat
Device management	Request and release devices, read, write, control devices	ioctl, read, write on device files, mmap of device memory
Information maintenance	Get or set time, date, system data, process attributes	getpid, time/clock_gettime, uname, sysinfo, getrusage
Communication	Create connections, send and receive messages, share memory	pipe, socket, send, recv, shmget/shm_open, mq_open
Protection	Control access to resources; set permissions	chmod, chown, umask, setuid, capget

5.2 What happens when a program calls read()

The path is shown in Figure 5.1 and explained step by step below, using the call `n = read(fd, buf, 100);`.

Figure 5.1 Step-by-step path of the read() system call

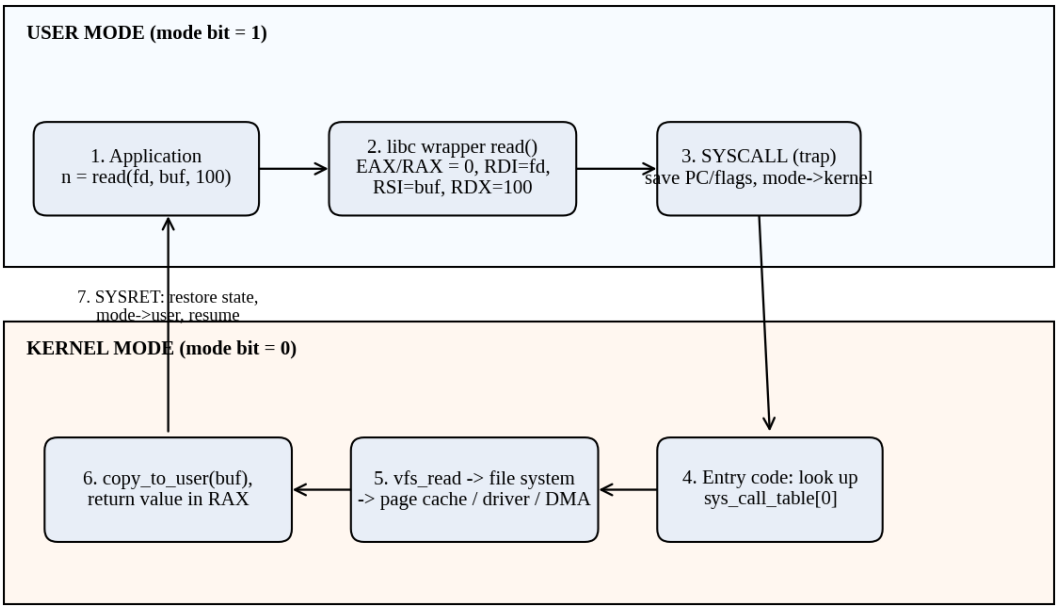


Figure 5.1 Path of a read() call from the application into the kernel and back

1. **The application calls the library function.** `read()` in the C library (glibc) is a small wrapper. The call to it is an ordinary function call; the program is still in user mode.
2. **The wrapper prepares the call.** On x86-64 Linux the system-call number (0 for read) is placed in register RAX, the arguments in RDI (file descriptor), RSI (buffer address) and RDX (count).
3. **Trap.** The wrapper executes the `syscall` instruction. This instruction is a *trap*: the CPU saves the user program counter and flags, switches the mode bit from user to kernel, and jumps to the kernel's fixed entry point. A user program cannot jump to arbitrary kernel code, only to the entry point the kernel registered, which is how protection is kept.
4. **Mode switch and dispatch.** The kernel entry code saves the other registers on the kernel stack, switches to the kernel stack and uses the number in RAX as an index into the system-call table to find `sys_read`.
5. **Kernel work.** `sys_read` looks up the file descriptor in the process's file table, checks the permission, and calls the virtual file system. If the data are in the page cache they are copied directly. Otherwise the driver issues a disk request, the process is put in the *waiting* state, the scheduler runs another process, and when the disk finishes it

interrupts the CPU (via DMA completion), the process becomes *ready* again. The kernel copies the data to the user buffer with `copy_to_user`, which also checks that the buffer address belongs to the process.

6. **Return.** The return value (the number of bytes read, or a negative error code) is placed in RAX; the registers are restored; the `sysret` instruction switches back to user mode and continues after the `syscall` instruction.
7. **The wrapper returns to the application.** If the value was negative, the wrapper stores the error code in `errno` and returns -1.

5.3 How parameters are passed to the kernel

Three methods are used.

- **Registers.** The simplest and fastest method; used by Linux on x86-64 for up to six parameters (RDI, RSI, RDX, R10, R8, R9). The limitation is the number of registers.
- **A block (table) in memory.** The parameters are stored in a block in memory and the address of the block is passed in a register. This is used when there are more parameters than registers, and by Linux for a few calls (for example `socketcall` on old 32-bit x86, and `clone3` which takes a pointer to a structure).
- **The stack.** The program pushes the parameters on the stack and the OS pops them (used by some systems such as older UNIX variants).

The block and stack methods do not limit the number or length of parameters. The kernel never trusts a pointer: it validates that the address belongs to the caller and copies the data between spaces.

5.4 Evidence: strace output from a real program

The program `t05_demo.c` opens a file, reads up to 100 bytes, writes them to the screen and closes the file.

Listing: t05_demo.c

```
/* Task 5: a tiny program whose system calls are easy to follow in the trace. */
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <unistd.h>
int main(void)
{
    char buf[100];
    int fd = open("/etc/hostname", O_RDONLY);     /* file management */
    ssize_t n = read(fd, buf, sizeof buf);        /* the read() call */
    write(1, buf, (size_t)n);                      /* device (stdout) */
    close(fd);
    return 0;
}
```

On the student's machine the command `strace ./t05_demo` shows lines such as `openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3`, `read(3, "vm\n", 100) = 3`, `write(1, "vm\n", 3) = 3`, `close(3) = 0`. In the sandbox used for this report strace was not installed, so a small tracer, `mystrace.c`, was written with the `ptrace` system call. It stops the child at every system-call entry and exit, which is exactly what strace does. The tracer is part of the repository. Its output for the program is shown below (arguments are printed as raw numbers by this simple tool, whereas strace decodes them).

Listing: mystrace.c

```
/* A tiny strace: prints every system call (number and name) made by a child, and a  \
      -c style summary.
   Usage: ./mystrace [-c] program args...   (x86-64 Linux only; uses ptrace) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/ptrace.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <sys/syscall.h>

static const char *name(long n)
{
    switch (n) {
    case SYS_read: return "read"; case SYS_write: return "write"; case SYS_open: ret \
      urn "open";
    case SYS_openat: return "openat"; case SYS_close: return "close"; case SYS_fstat \
      : return "fstat";
    case SYS_newfstatat: return "newfstatat"; case SYS_mmap: return "mmap"; case SYS \
      _munmap: return "munmap";
    case SYS_mprotect: return "mprotect"; case SYS_brk: return "brk"; case SYS_execv \
      e: return "execve";
    case SYS_exit_group: return "exit_group"; case SYS_access: return "access"; case \
       SYS_pread64: return "pread64";
    case SYS_arch_prctl: return "arch_prctl"; case SYS_set_tid_address: return "set_ \
      tid_address";
    case SYS_set_robust_list: return "set_robust_list"; case SYS_rseq: return "rseq" \
      ; case SYS_prlimit64: return "prlimit64";
    case SYS_getrandom: return "getrandom"; case SYS_unlink: return "unlink"; case S \
      YS_lseek: return "lseek";
    case SYS_ioctl: return "ioctl"; case SYS_fork: return "fork"; case SYS_clone: re \
      turn "clone";
    case SYS_wait4: return "wait4"; case SYS_getpid: return "getpid"; case SYS_readl \
      ink: return "readlink";
    default: return NULL;
    }
}
int main(int argc, char **argv)
{
```

```
    int summary = argc > 2 && !strcmp(argv[1], "-c");
    char **prog = argv + (summary ? 2 : 1);
    pid_t pid = fork();
    if (pid == 0) { ptrace(PTRACE_TRACEME, 0, 0, 0); execvp(prog[0], prog); _exit(12 \
      7); }
    int st, in_call = 0; long count[512] = {0};
    waitpid(pid, &st, 0);
    ptrace(PTRACE_SETOPTIONS, pid, 0, PTRACE_O_TRACESYSGOOD);
    for (;;) {
        ptrace(PTRACE_SYSCALL, pid, 0, 0);
        waitpid(pid, &st, 0);
        if (WIFEXITED(st)) break;
        if (WIFSTOPPED(st) && (WSTOPSIG(st) & 0x80)) {
            struct user_regs_struct r; ptrace(PTRACE_GETREGS, pid, 0, &r);
            if (!in_call) {
                in_call = 1;
                long n = (long)r.orig_rax; if (n >= 0 && n < 512) count[n]++;
                if (!summary) {
                    const char *nm = name(n);
                    if (nm) fprintf(stderr, "%s(%#llx, %#llx, %llu)", nm, r.rdi, r.r \
      si, r.rdx);
                    else fprintf(stderr, "syscall_%ld(...)", n);
                }
            } else {
                in_call = 0;
                if (!summary) fprintf(stderr, " = %lld\n", (long long)r.rax);
            }
        }
    }
    if (summary) {
        fprintf(stderr, "%-18s %8s\n", "syscall", "calls");
        long tot = 0;
        for (int i = 0; i < 512; i++) if (count[i]) { const char *nm = name(i); char \
       b[32]; if (!nm) { snprintf(b, sizeof b, "syscall_%d", i); nm = b; } fprintf(s \
      tderr, "%-18s %8ld\n", nm, count[i]); tot += count[i]; }
        fprintf(stderr, "%-18s %8ld\n", "total", tot);
    }
    return 0;
}
```

$ ./mystrace ./t05_demo 2>&1 | tail -9

```
mprotect(0x7f2372520000, 0x2000, 1) = 0
prlimit64(0, 0x3, 0) = 0
munmap(0x7f23724d3000, 0xc243, 0) = 0
openat(0xffffff9c, 0x556a8b254004, 0) = 3
read(0x3, 0x7ffc66d41100, 100) = 3
write(0x1, 0x7ffc66d41100, 3)vm
 = 3
close(0x3, 0x7ffc66d41100, 3) = 0
exit_group(0, 0xffffffffffffff88, 231)
```

Reading the trace: the lines before `openat` are the start-up of the dynamic loader (memory mapping the C library, setting thread data, protecting memory). Then `openat(...) = 3` returns the file descriptor 3 (0, 1 and 2 are standard input, output and error); `read(0x3, buf, 100) = 3` returns the number of bytes read (the file contains 3 bytes: "vm" and newline); `write(0x1, buf, 3) = 3` writes them to standard output; `close(0x3)` releases the descriptor; and `exit_group` ends the process. The whole program contains five visible calls of its own and about 45 more calls generated by the C library and the dynamic loader. This shows that even a very small program asks the kernel for many services.

The `-c` option summarises the number of calls by type. On the student's machine the equivalent is `strace -c ./t06_copy`. The result of the tracer for the copy program of Task 6 is:

```
$ ./mystrace -c ./t06_copy 2>&1 | head -8
```

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
syscall               calls
read                  10027
write                 10027
close                     9
fstat                     4
```

5.5 Alternative concept: fopen/fread/fwrite instead of open/read/write

What the solution would be. The program calls the C library's buffered functions. The library keeps a buffer (4096 bytes by default for files, on most systems equal to the file system's preferred block size `st_blksize`) and calls `read()` only when the buffer is empty and `write()` only when it is full.

Number of system calls. For a file of n bytes copied byte by byte (`fgetc`/`fputc`):

- direct `read`/`write` with one byte at a time: n read calls + n write calls = 2n system calls, O(n);
- library functions with the default buffer of B = 4096 bytes: about n/B read calls + n/B write calls = 2n/B system calls, still O(n) but with a constant 4096 times smaller;
- direct `read`/`write` with an application buffer of b bytes: 2n/b system calls.

The program `t05_stdio.c` copies 10,000 bytes one character at a time with `fgetc` and `fputc`. The tracer reports the number of read and write system calls.

Listing: t05_stdio.c

```
/* Task 5 alternative: the same copy loop with fread/fwrite - the library buffers 40 \
      96 bytes. */
#include <stdio.h>
int main(void)
{
    FILE *f = fopen("in.dat", "w");
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);
    FILE *in = fopen("in.dat", "r"), *out = fopen("out.dat", "w");
    int c;
    while ((c = fgetc(in)) != EOF) fputc(c, out);    /* 10,000 library calls */
    fclose(in); fclose(out);
    return 0;
}
```

```
$ ./mystrace -c ./t05_stdio 2>&1 | grep -E "^(read|write|total)"
```

```
read                     5
write                    6
total                   51
```

The numbers show about 3 read calls and 3 write calls for 10,000 library calls (10,000 / 4096 = 2.44, rounded up to 3, plus a few calls made by the loader), instead of 10,000 + 10,000 calls without the library's buffer. Only the user-mode library calls are repeated 10,000 times, and they cost a few nanoseconds each.

Extra buffer space. The library allocates one buffer per open stream: 4096 bytes (or 8192 on some systems) for each file, so about 8 KB for the two streams in the copy program. The space is O(B) per stream and is independent of the file size n. For a program that keeps 10,000 files open (not realistic for a monitor) this would be 40 MB, which is why very small embedded systems may prefer direct calls with a chosen buffer.

Trade-offs. The library is easier to use, portable, and fast for small, repeated reads and writes. But it can hide data in a buffer: if the program crashes before `fflush` or `fclose` the buffered bytes are lost (this is exactly the effect shown in Task 7 where the buffer is copied by `fork`). It also performs a copy from the library buffer to the user's own buffer, so one more memory copy occurs. For a record store that needs control over when data reach the disk, the database uses `write` followed by `fsync` directly, which is the choice made in the final plan.

5.6 Evidence summary for Task 5

The categories of system calls, the seven steps of read(), the three parameter-passing methods and a real trace of a program are given, together with a measured comparison of the number of system calls with and without library buffering.

TASK 6 – Evaluate the Cost of System Calls (4 Marks)

6.1 Why a system call costs much more than a function call

An ordinary function call is a jump to a known address and back: the processor pushes a return address, jumps, executes, and returns, taking about 1 to 5 clock cycles (roughly 1 ns). A system call does much more work.

1. The `syscall` instruction itself flushes parts of the CPU pipeline and switches privilege level (tens of cycles).
2. The kernel saves all user registers, switches stacks and, since the Meltdown vulnerability, usually also switches page tables (kernel page-table isolation), which can invalidate TLB entries.
3. The kernel validates every argument (file descriptor, pointer, length) because it cannot trust the user.
4. The real work is done (look up the file, check permissions, copy data, possibly wait for the disk).
5. On return the registers are restored and the pipeline refilled. Caches and the branch predictor are partly polluted by kernel code, so the user program runs slower for a short time after the call.

The total is typically 100 to 1,000 ns for the cheapest calls (such as `getppid`) and many microseconds for calls that touch devices. The measurements in this report show the effect directly: copying 4 MB one byte at a time needed 4,194,304 read calls and 4,194,304 write calls and took 2.33 s, about 280 ns per system call (556 ns per read-write pair) on this virtual machine, with 80 % of the time spent in the kernel (sys time 1.85 s against user time 0.45 s).

6.2 The program: copying a file with read() and write()

The completed program for the assignment is listed below. TODO 1 was replaced by a loop that calls `read()` with the chosen buffer size, counts every call (including the last one, which returns 0 at end of file), and writes exactly the number of bytes that were read, repeating the write if the kernel accepted fewer bytes than requested (a *short write*).

Listing: t06_copy.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
```

```
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    /* TODO 1 (completed) */
    for (;;) {
        n = read(in, buf, bufsize);  /* one system call per iteration    */
        calls++;                     /* count every read(), also the last */
        if (n <= 0) break;           /* 0 = end of file, -1 = error     */
        ssize_t done = 0;
        while (done < n) {           /* write exactly the n bytes read   */
            ssize_t w = write(out, buf + done, (size_t)(n - done));
            if (w < 0) { close(in); close(out); return -1; }
            done += w;
        }
    }
    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w"); /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);
    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

The program was compiled with `gcc -std=c11 -Wall -Wextra -pthread -O2 t06_copy.c -o t06_copy` without warnings and executed:

`$ ./t06_copy`

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

The output is identical to the expected output. The numbers can be explained directly: a file of 10,000 bytes read with a buffer of 1 byte needs 10,000 successful calls plus 1 call that

returns 0 (end of file), giving 10,001. With 512 bytes the file needs ceil(10000/512) = 20 successful calls and the final one, giving 21. With 4096 bytes it needs ceil(10000/4096) = 3 successful calls and the final one, giving 4. In general the number of read calls is ceil(n/b) + 1.

Comparison with the strace -c count

The assignment asks to run the program under `strace -c` and compare. The same count was made with the tracer from Task 5.

```
$ ./mystrace -c ./t06_copy 2>&1 | grep -E "^(read|write|total|openat|close)"
```

```
read                10027
write               10027
close                   9
openat                  9
total               20103
```

The program printed 10,001 + 21 + 4 = 10,026 read calls. The trace reports 10,027 read calls: one more, because the dynamic loader reads the ELF header of the C library once, before `main` starts (this is the `read(3, "\177ELF...", 832)` that every dynamically linked program shows). The number of writes is 10,000 + 20 + 3 = 10,023 for the copies plus a few writes made by `printf` and by `fputc` when the test file was created (the library flushes its 4096-byte buffer 3 times for the 10,000 bytes). The count of calls reported by the tool therefore confirms the count printed by the program, and it shows the extra calls hidden in the library.

6.3 Measurements for a larger file

To see the effect on time, the program `t06_bench.c` copies a 4 MB file with several buffer sizes.

Listing: t06_bench.c

```
/* Tasks 6 and 10: copy a 4 MB file with read()/write() (several buffer sizes),
   with fread()/fwrite() and with mmap(); report calls and times. */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/resource.h>

#define FILESZ (4L * 1024 * 1024)

static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
```

```
      urn t.tv_sec + t.tv_nsec / 1e9; }
static double tv(struct timeval x, struct timeval y) { return (x.tv_sec - y.tv_sec)  \
      + (x.tv_usec - y.tv_usec) / 1e6; }

static long copy_rw(const char *s, const char *d, size_t bs)
{
    char *buf = malloc(bs);
    int in = open(s, O_RDONLY), out = open(d, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    long calls = 0; ssize_t n;
    while ((n = read(in, buf, bs)) > 0) { calls++; if (write(out, buf, n) != n) brea \
      k; }
    close(in); close(out); free(buf); return calls;
}
static long copy_stdio(const char *s, const char *d, size_t bs)
{
    FILE *in = fopen(s, "r"), *out = fopen(d, "w");
    char *buf = malloc(bs); long calls = 0; size_t n;
    while ((n = fread(buf, 1, bs, in)) > 0) { calls++; fwrite(buf, 1, n, out); }
    fclose(in); fclose(out); free(buf); return calls;
}
static void copy_mmap(const char *s, const char *d)
{
    int in = open(s, O_RDONLY), out = open(d, O_RDWR | O_CREAT | O_TRUNC, 0644);
    struct stat st; fstat(in, &st);
    if (ftruncate(out, st.st_size)) return;
    void *a = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, in, 0);
    void *b = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, out, 0);
    memcpy(b, a, st.st_size);
    munmap(a, st.st_size); munmap(b, st.st_size); close(in); close(out);
}
#define MEASURE(label, calls_expr) do { struct rusage a, b; getrusage(RUSAGE_SELF, & \
      a); double t0 = now(); \
    long c = (calls_expr); double t1 = now(); getrusage(RUSAGE_SELF, &b); \
    printf("%-24s %10ld %9.4f %9.4f %9.4f\n", label, c, t1 - t0, tv(b.ru_utime, a.ru \
      _utime), tv(b.ru_stime, a.ru_stime)); fflush(stdout); } while (0)

int main(void)
{
    char *chunk = malloc(1 << 20); memset(chunk, 'x', 1 << 20);
    int fd = open("src.dat", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    for (long i = 0; i < FILESZ >> 20; i++) if (write(fd, chunk, 1 << 20) < 0) retur \
      n 1;
    close(fd);
    printf("%-24s %10s %9s %9s %9s\n", "method (4 MB file)", "read calls", "real(s)" \
      , "user(s)", "sys(s)");
    size_t sizes[] = {1, 16, 512, 4096, 65536, 1048576};
    for (int i = 0; i < 6; i++) {
        char nm[40]; snprintf(nm, sizeof nm, "read/write b=%zu", sizes[i]);
        MEASURE(nm, copy_rw("src.dat", "dst.dat", sizes[i]));
    }
    size_t fs[] = {16, 4096, 65536};
```

```
    for (int i = 0; i < 3; i++) {
        char nm[40]; snprintf(nm, sizeof nm, "fread/fwrite b=%zu", fs[i]);
        MEASURE(nm, copy_stdio("src.dat", "dst.dat", fs[i]));
    }
    MEASURE("mmap + memcpy", (copy_mmap("src.dat", "dst.dat"), 0L));
    unlink("src.dat"); unlink("dst.dat");
    return 0;
}
```

$ ./t06_bench

method (4 MB file)	read calls	real(s)	user(s)	sys(s)
read/write b=1	4194304	2.3343	0.4494	1.8544
read/write b=16	262144	0.1392	0.0428	0.0928
read/write b=512	8192	0.0121	0.0000	0.0076
read/write b=4096	1024	0.0073	0.0030	0.0006
read/write b=65536	64	0.0062	0.0005	0.0020
read/write b=1048576	4	0.0030	0.0003	0.0013
fread/fwrite b=16	262144	0.0097	0.0001	0.0084
fread/fwrite b=4096	1024	0.0073	0.0008	0.0031
fread/fwrite b=65536	64	0.0060	0.0005	0.0019
mmap + memcpy	0	0.0041	0.0004	0.0014

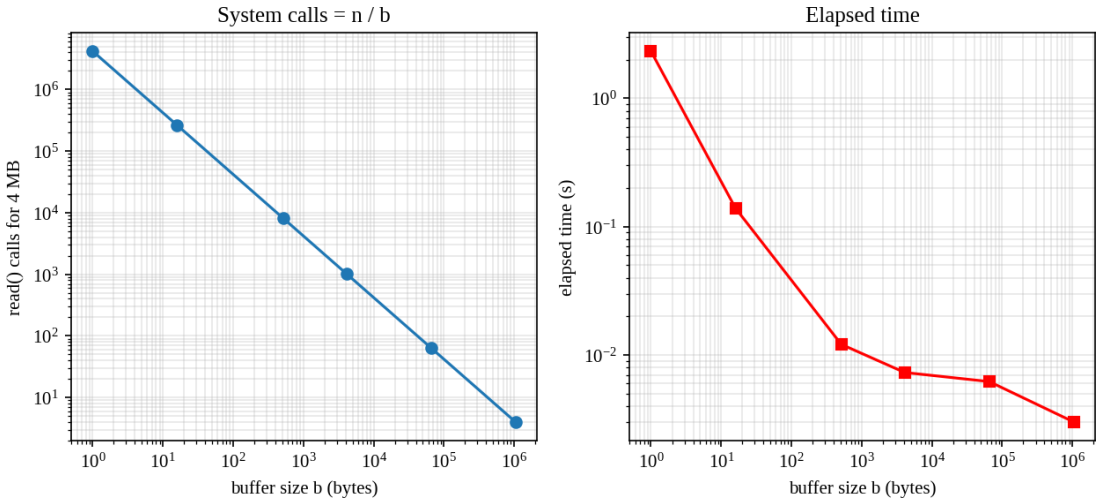


Figure 6.1 Number of read() calls and elapsed time against buffer size (4 MB file)

Interpretation

- The number of calls falls in inverse proportion to the buffer size, as n/b predicts (4,194,304 for b = 1, 262,144 for b = 16, 8,192 for b = 512, 1,024 for b = 4096, 64 for 64 KB and 4 for 1 MB).
- The time falls steeply at first (2.33 s to 0.14 s to 0.012 s) and then flattens (0.0073 s for 4 KB, 0.0062 s for 64 KB, 0.0030 s for 1 MB). The curve has the shape T(b) = c_call × (n/b) + c_byte × n: the first term is the per-call overhead, which shrinks with b; the second is the per-byte cost of copying and of the device, which does not depend on b. Once the first term is small compared with the second, a bigger buffer gains

almost nothing.

- Beyond a certain size a bigger buffer can even harm performance, because the buffer no longer fits in the CPU cache (L1 is 32 to 64 KB, L2 256 KB to 2 MB) and the data written by `read` must be re-read from main memory by `write`.
- The library functions behave as predicted. With a 16-byte user buffer the stdio program needed the same 262,144 calls at library level but only 0.0097 s, compared with 0.139 s for direct calls with a 16-byte buffer. The reason is that the stdio library internally reads and writes 4096 bytes at a time, so the kernel sees only 1,024 calls and not 262,144.
- Memory mapping took 0.0041 s with only a few system calls (open, fstat, ftruncate, two mmap, two munmap, close).

Best buffer size for the backup program

The backup program should use a buffer between 64 KB and 1 MB. The measurements show that 4 KB is the point where the per-call overhead has become small (about 1,000 calls for 4 MB, 0.0073 s), 64 KB reduces this by another factor of 16 with almost no additional cost, and 1 MB gives the lowest time. A larger buffer than 1 MB gives no further gain on this machine. For a real hospital backup writing to a RAID array or network storage a buffer of 256 KB to 1 MB is also a good match for the RAID stripe size and the network round trip. The choice of **256 KB** is recommended: it is large enough to make the system-call overhead negligible (4,096 calls for 1 GB), small enough to stay in the L2 cache, and the memory use is trivial.

6.4 Alternative concept: memory-mapped files (mmap)

What the solution would be. Instead of copying data through a user buffer by `read` and `write`, the backup maps the source file and the destination file into its address space with `mmap()` and copies with `memcpy()`. The code is:

```
int in  = open(src, O_RDONLY);
int out = open(dst, O_RDWR | O_CREAT | O_TRUNC, 0644);
fstat(in, &st);  ftruncate(out, st.st_size);
void *a = mmap(NULL, st.st_size, PROT_READ,  MAP_PRIVATE, in,  0);
void *b = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, out, 0);
memcpy(b, a, st.st_size);
munmap(a, st.st_size);  munmap(b, st.st_size);
```

Number of system calls. The read/write copy needs 2n/b + O(1) calls (O(n/b)). The mmap copy needs a constant number of calls (about 8) independent of n, i.e. O(1). The data movement happens through page faults: touching a page of the mapping causes a page fault that the kernel serves, so the cost of one fault per 4 KB page replaces the cost of one system call per buffer. Page faults are cheaper than system calls for small pages when the page is

already in the page cache, but not free, so the total kernel work is still O(n / page size). The advantage is mainly that the file data is not copied into the process by the kernel.

Copying between kernel and user space. With `read`, the kernel copies each block from the page cache into the user buffer, and `write` copies it from the user buffer back into the kernel. That is two copies per byte (plus the device transfer). With `mmap` the page-cache page itself is mapped into the process; `memcpy` then copies source page to destination page, which is one copy per byte in user space and no kernel copy. (The best method is `copy_file_range` or `sendfile`, in which the kernel copies directly with no user-space visit at all.)

Memory for very large files. The read/write method uses O(b) memory, independent of the file size. The mmap method uses virtual address space O(n), which is a problem on a 32-bit system for a 5 GB file, and on 64-bit it is only address space, because pages are brought in on demand and the kernel can discard clean pages under memory pressure. But the resident set can grow as large as the page cache allows, and the program cannot easily handle an I/O error: a read error on a mapped file arrives as a signal (SIGBUS) rather than a return value, which is harder to handle correctly. For a backup tool that must report errors clearly, the buffer method with `read`/`write` (or `copy_file_range`) is therefore still recommended.

6.5 Evidence summary for Task 6

Measured system-call counts (10,001 / 21 / 4 for the 10,000-byte file; 4,194,304 to 4 for the 4 MB file) and times for six buffer sizes support the recommendation of a 256 KB buffer for the hospital backup program.

TASK 7 – Evaluate Process Control System Calls (4 Marks)

7.1 What fork(), exec(), wait() and exit() do

- **fork()** creates a new process, the child, which is an almost exact copy of the parent: the same code, a copy of the data, stack and heap (implemented with copy-on-write so pages are really copied only when one process writes to them), the same open files, and the same program counter. It returns twice: it returns the child's process ID in the parent and 0 in the child (and -1 if it fails). The child gets a new process ID.
- **exec()** (the family `execl`, `execlp`, `execv`, `execve` and so on) replaces the current process image with a new program. The process ID, the open file descriptors (unless marked close-on-exec) and the environment persist, but the code, data, heap and stack are replaced by those of the new program. If `exec` succeeds it never returns; if it fails it returns -1, and the caller must handle it (the shell code calls `exit(127)`).

- **wait() / waitpid()** blocks the parent until a child terminates, and returns the child's status, from which the macros `WIFEXITED` and `WEXITSTATUS` extract the exit code. It also *reaps* the child, that is, it lets the kernel free the child's process-table entry.
- **exit() / _exit()** terminates the calling process. The kernel closes its files, frees its memory, sends SIGCHLD to the parent and keeps a small entry (the exit status) until the parent calls wait. `exit()` first runs the functions registered with `atexit` and flushes the C library buffers; `_exit()` goes straight to the kernel.

7.2 How a shell runs a command and obtains the exit status

Figure 7.1 How a shell runs a command: fork - exec - wait - exit

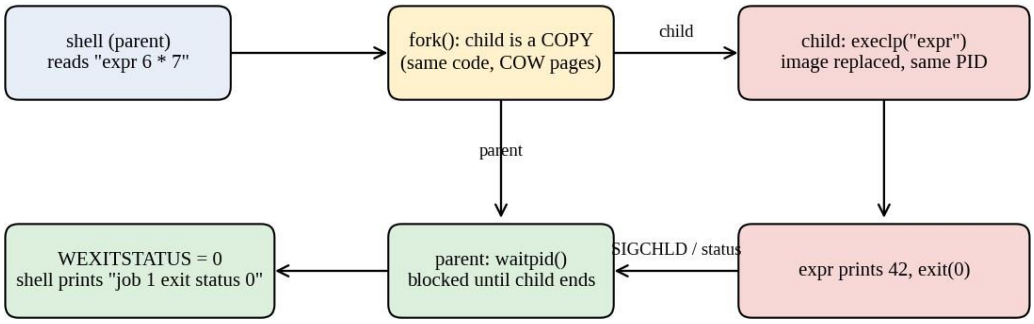


Figure 7.1 Shell running a command with fork, exec and wait

The shell reads a line, parses it, then (for an external command) calls `fork()`. In the child it calls `execlp()` or `execvp()` with the command name and arguments; the kernel searches the `PATH`, loads the program and starts it. In the parent the shell calls `waitpid(pid, &status, 0)` and sleeps until the child terminates. The `status` integer contains the exit code in bits 8 to 15, which `WEXITSTATUS(status)` extracts. The shell stores the value in the variable `$?`, and it is used by `&&`, `||` and `if`. By convention 0 means success and any other value means an error; 127 means "command not found", 126 "found but not executable", and 128+N "killed by signal N".

7.3 The completed program

TODO 1 was replaced by code that forks; in the child executes `expr 6 * 7` and calls `exit(127)` if `exec` fails; and in the parent waits and prints the status. The second half of the given code uses the variables `pid` and `status` without declaring them, so they are declared in the TODO block (the assignment says to replace the TODO with working code, and a declaration is needed for the program to compile).

Listing: t07_shell.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout); /* empty the buffer before fork() */

    /* TODO 1 (completed) */
    pid_t pid;
    int status;
    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        exit(127);                      /* only reached if exec failed */
    }
    waitpid(pid, &status, 0);
    printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3); /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

`$ ./t07_shell`

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

The output is the same as the expected output. The "42" is printed by the `expr` program itself, using the same standard output that it inherited from the shell through `fork`; `job 1 exit status 0` is printed after the child has finished, because the parent waited. The second job uses `_exit(3)` in the child, so the parent obtains status 3.

7.4 Why stdout is flushed before each fork()

The C library buffers standard output. When output goes to a terminal it is *line buffered* (the buffer is written at every newline); when it goes to a file or a pipe it is *fully buffered*

(written when 4096 or more bytes accumulate, or at exit). `fork()` copies the whole address space of the parent, including the library's buffer. If the buffer contains text that has not been written, both the parent and the child have a copy of that text and *each of them will write it* when it flushes. Flushing before `fork()` empties the buffer so that nothing is duplicated.

Experiment: remove the second fflush and use exit(3)

The second `fflush(stdout)` was removed, `_exit(3)` was changed to `exit(3)` (file `t07_noflush.c`), and the output was redirected to a file so that the buffer is fully buffered.

```
$ diff t07_shell.c t07_noflush.c
```

```
25d24
<     fflush(stdout);
27c26
<     if (pid == 0) _exit(3); /* a child that fails with code 3 */
---
>     if (pid == 0) exit(3); /* a child that fails with code 3 */
```

```
$ ./t07_noflush > out_noflush.txt; cat out_noflush.txt
```

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

"shell: running job 2" now appears twice (and, because of the shared buffer, "job 1 exit status 0" is also repeated). The explanation, step by step: (1) the redirected standard output is fully buffered, so the lines printed so far after the first flush, namely "job 1 exit status 0" and "shell: running job 2", sit in the buffer; (2) `fork()` copies the buffer into the child; (3) the child calls `exit(3)`, which, unlike `_exit(3)`, runs the library clean-up and flushes the stdio buffers, so the child writes the two copied lines; (4) the parent later flushes its own copy at exit, writing the lines again. When the output is a terminal the same program shows no duplicates, because the line-buffered stream was flushed at each newline before `fork`. The lesson is that a child created by `fork` that does not call `exec` should leave with `_exit()`, and that buffered output must be flushed before `fork()`.

7.5 Zombie and orphan processes

A **zombie** is a process that has terminated but whose parent has not yet called `wait()`. It has released its memory but its process-table entry (with the exit status) stays, so that the parent can still read the status. It uses no CPU or memory except that entry, but a parent that creates children and never waits fills the process table and no new process can be

created. A **orphan** is a process whose parent has terminated while the process is still running. The kernel *re-parents* orphans to the `init` process (PID 1, or a subreaper), which regularly calls `wait()` and so cleans them up. The program `t07_zombie.c` shows a real zombie:

Listing: t07_zombie.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
/* Creates a zombie (child exits, parent has not yet called wait) and shows its stat \
      e in /proc. */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) _exit(0);
    sleep(1);                                   /* child is now a zombie */
    char cmd[64];
    snprintf(cmd, sizeof cmd, "grep -E 'Name|State' /proc/%d/status", pid);
    printf("child %d after exit, before wait():\n", pid); fflush(stdout);
    if (system(cmd)) {}
    waitpid(pid, NULL, 0);                      /* reaping removes the zombie */
    printf("after waitpid(): "); fflush(stdout);
    snprintf(cmd, sizeof cmd, "test -e /proc/%d && echo still there || echo entry re \
      moved", pid);
    if (system(cmd)) {}
    return 0;
}
```

```
$ ./t07_zombie
```

```
child 20439 after exit, before wait():
Name:    t07_zombie
State:    Z (zombie)
after waitpid(): entry removed
```

The state `Z (zombie)` is shown by `/proc/<pid>/status` after the child has exited and before the parent calls `waitpid`, and the entry disappears after `waitpid`. In the hospital database server the shell scripts that start backup jobs must therefore wait for their children (or ignore SIGCHLD with `SA_NOCLDWAIT`), otherwise a long-running daemon accumulates zombies.

7.6 Why fork() followed by exec() and not one create-process call

UNIX separates *creating a process* from *loading a program*. The separation gives the shell a window between the two calls in which the child, still running the shell's code, can set up its environment: it can redirect standard input and output (`open` then `dup2` onto descriptors

0 and 1), build a pipe for a pipeline `a | b`, change directory, set limits and user ID, close unneeded descriptors and set signal handlers, and only then `exec` the program, which inherits that environment. A single call such as Windows `CreateProcess` must take all these options as parameters, which makes it a large, complex call. The disadvantage of fork-then-exec is that `fork` appears to copy the whole address space just before throwing it away at `exec`; copy-on-write and `vfork` reduce that cost.

7.7 Alternative concept: posix_spawn() or threads instead of fork + exec

What the solution would be. With `posix_spawn()` the shell asks the kernel to create a process running the new program directly, passing a list of file actions (redirections) and attributes, so that there is no intermediate copy of the parent. With threads, each command would be run by `pthread_create` inside the shell process, which is only possible for code that is already inside the shell (builtin commands) and not for an external program, which needs its own address space.

Time to start 1,000 commands. The program `t10_create.c` (listed in Task 10) measured the three methods on this machine for the external command `true`.

Method (1,000 repetitions, command true)	Parent heap 0 MB	Parent heap 200 MB
fork + exec + wait	880 us each	3,682 us each
posix_spawn + wait	681 us each	869 us each
fork + exit only (no exec)	59.5 us each	2,121 us each
thread create + join	10.3 us each	13.8 us each

Memory copied or shared. A `fork` of a parent with a large address space must duplicate the page tables, which takes time proportional to the size of the parent's mapped memory, O(M). With a 200 MB parent the fork cost rose from 59.5 µs to 2,121 µs (35 times) and fork + exec from 880 µs to 3,682 µs. `posix_spawn` (implemented with `clone(CLONE_VM | CLONE_VFORK)` in glibc) shares the parent's memory until exec and its cost does not grow with the parent's size (681 µs to 869 µs). Threads share *everything* (code, heap, globals, open files) and need only a new stack and thread structure; they take 10 to 14 µs.

Isolation. A forked or spawned process has its own address space: if it crashes or is compromised it cannot corrupt the shell, and the kernel can account for its memory and kill it independently. A thread shares the address space with the shell, so one bad command can corrupt the shell's data or crash the shell, and a command cannot be killed safely without risking the whole process. A command shell on the database server must isolate commands, so threads are rejected for running external commands.

Judgement. `fork + exec` is retained for a general shell because of its flexibility; `posix_spawn` is a better choice for a large multithreaded server (such as a monitoring daemon with a big heap) that starts many external programs, since its cost does not depend on the size of the parent. Threads are right for concurrency *inside* one program.

7.8 Evidence summary for Task 7

A working program that runs commands like a shell, the process diagram (Figure 7.1), the flush experiment, the zombie demonstration and the measurements for fork, spawn and threads are given.

TASK 8 – Evaluate CPU States and Protection (3 Marks)

8.1 User mode, kernel mode, privileged instructions and the mode bit

Hardware provides at least two modes of operation: **user mode** and **kernel (supervisor, privileged) mode**. A *mode bit* in a CPU register indicates the current mode (on x86 the current privilege level in the low bits of the code segment selector: ring 0 for the kernel and ring 3 for applications). The operating system runs in kernel mode, and applications run in user mode. Certain instructions are **privileged** and can be executed only in kernel mode: I/O instructions, changing the page-table base register, enabling or disabling interrupts, halting the CPU, loading the timer, modifying the mode bit itself. If a user program tries to execute one, the hardware raises an exception (a general-protection fault) and the OS terminates the program. The mode changes from user to kernel only through an interrupt, a trap (system call) or an exception, each of which loads a program counter that the kernel set up, and from kernel to user only by a special return instruction executed by the kernel. Consequently a user program cannot gain kernel privileges by its own action.

8.2 Interrupts, traps and the timer interrupt

An **interrupt** is a signal from hardware (a disk controller, a network card, the sensor of the monitor) that makes the CPU stop what it is doing, save its state and jump to an *interrupt service routine* found through the interrupt vector table. After handling, the CPU resumes the interrupted program. A **trap** or **exception** is generated by software or by the CPU itself: a system call, a division by zero, an invalid memory reference, a page fault. Interrupts are asynchronous (they can arrive at any time); traps are synchronous (caused by the current instruction).

The **timer interrupt** is the mechanism that stops a program from keeping the CPU forever. Before giving control to a user program the OS sets a hardware timer (for example 1 ms or 4 ms, depending on the kernel configuration); when the timer expires it interrupts the CPU,

the kernel regains control, and the scheduler may choose another process. Without a timer, a program with an infinite loop would hang the monitor forever. The timer also keeps the system time and drives timeouts.

8.3 What would happen to the monitors if any program could disable interrupts

If any user program could execute the instruction that disables interrupts (`cli` on x86), any bug or malicious code that executed it and then went into a loop would:

1. stop the **timer interrupt**, so the scheduler would never run again and the other tasks, including the alarm task, would starve;
2. stop **device interrupts**: the sensor's data-ready interrupt would not be seen, the readings would be lost and no alarm could be raised within 10 ms;
3. stop the **network interrupts**, so the monitor could not report to the nurses' station or accept a command;
4. leave the system hanging with the only recovery being a hardware watchdog reset or the power switch.

Because the alarm deadline is 10 ms, even a legitimate program that disabled interrupts for 20 ms would break the requirement. That is why `cli` is privileged, why the kernel itself disables interrupts for as short a time as possible (microseconds), and why a real-time kernel has a bounded *maximum interrupt-disabled time*. A hardware watchdog timer (a separate counter that resets the CPU unless it is regularly kicked by the software) is also used on the monitors as the last defence.

8.4 How the hardware stops one program from reading another program's memory

The CPU's **memory-management unit (MMU)** translates every virtual address generated by a program into a physical address using page tables that the kernel sets up for each process. A process can only generate addresses that its own page table maps. The page-table entry also contains permission bits (read, write, execute, user/supervisor). When the program uses an address that is not mapped, or writes to a read-only page, or a user program touches a supervisor page, the MMU raises a page fault and the kernel terminates the program (on Linux with SIGSEGV). Because the page-table base register can be loaded only in kernel mode, a program cannot change its own mapping. On simpler monitors that have no MMU the hardware offers a *memory-protection unit* (MPU) with base and limit registers for a few regions, which has the same effect for a small number of tasks. In the base-and-limit scheme every address is compared with the process's base and limit in hardware; an address outside the range causes a trap.

8.5 Experiment: polling versus interrupt-driven detection

The program `t08_polling.c` simulates an abnormal reading that appears at a known time (about 203 ms after the start). It detects it by polling every 50 ms, 10 ms and 1 ms, by busy polling, and by waiting for a timer signal (a software analogue of an interrupt: the process sleeps and the kernel wakes it).

Listing: t08_polling.c

```
/* Task 8: polling versus interrupt (signal) driven detection of an abnormal event.
   A timer thread-less simulation: the "sensor" raises an event at a random time; we \
       compare
   (a) polling every P ms, (b) waiting for a signal (SIGALRM), and report the CPU ti \
      me used. */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <sys/time.h>
#include <time.h>
#include <unistd.h>
#include <sys/resource.h>
static volatile sig_atomic_t flag = 0, got = 0;
static struct timespec t_event, t_seen;
static void on_alarm(int s) { (void)s; clock_gettime(CLOCK_MONOTONIC, &t_seen); got  \
      = 1; }
static double ms(struct timespec a, struct timespec b) { return (b.tv_sec - a.tv_sec \
      ) * 1e3 + (b.tv_nsec - a.tv_nsec) / 1e6; }
static double cpu_ms(void) { struct rusage r; getrusage(RUSAGE_SELF, &r); return r.r \
      u_utime.tv_sec * 1e3 + r.ru_utime.tv_usec / 1e3 + r.ru_stime.tv_sec * 1e3 + r. \
      ru_stime.tv_usec / 1e3; }
static void sleep_ms(double m) { struct timespec t = {(time_t)(m / 1000), (long)((m  \
      - 1000 * (long)(m / 1000)) * 1e6)}; nanosleep(&t, NULL); }
int main(void)
{
    signal(SIGALRM, on_alarm);
    double periods[] = {50, 10, 1, 0};    /* 0 = busy polling */
    printf("%-18s %14s %14s\n", "method", "delay (ms)", "CPU used (ms)");
    for (int k = 0; k < 4; k++) {
        double P = periods[k], c0 = cpu_ms();
        struct timespec st; clock_gettime(CLOCK_MONOTONIC, &st);
        double event_at = 200.0 + 0.37 * (k + 1) * 10;       /* event at ~203.7 ms et \
      c. */
        flag = 0;
        for (;;) {
            struct timespec n; clock_gettime(CLOCK_MONOTONIC, &n);
            double el = ms(st, n);
            if (el >= event_at && !flag) { flag = 1; t_event = n; t_event.tv_nsec -= \
       (long)((el - event_at) * 1e6); }
            if (flag) { t_seen = n; break; }
            if (P > 0) sleep_ms(P);
```

```
        }
        double delay = ms(st, t_seen) - event_at;
        char nm[32]; if (P > 0) snprintf(nm, sizeof nm, "poll every %g ms", P); else \
       snprintf(nm, sizeof nm, "busy polling");
        printf("%-18s %14.3f %14.3f\n", nm, delay, cpu_ms() - c0);
    }
    /* interrupt-driven: ask the kernel to deliver SIGALRM at 203 ms, and sleep (no  \
      CPU used) */
    double c0 = cpu_ms(); struct timespec st; clock_gettime(CLOCK_MONOTONIC, &st);
    struct itimerval it = {{0, 0}, {0, 203000}}; setitimer(ITIMER_REAL, &it, NULL);
    while (!got) pause();
    printf("%-18s %14.3f %14.3f\n", "interrupt (SIGALRM)", ms(st, t_seen) - 203.0, c \
      pu_ms() - c0);
    return 0;
}
```

$ ./t08_polling

```
method                  delay (ms)  CPU used (ms)
poll every 50 ms            47.090          0.254
poll every 10 ms             5.730          0.810
poll every 1 ms              0.937          3.651
busy polling                 0.000        208.920
interrupt (SIGALRM)          0.146          0.063
```

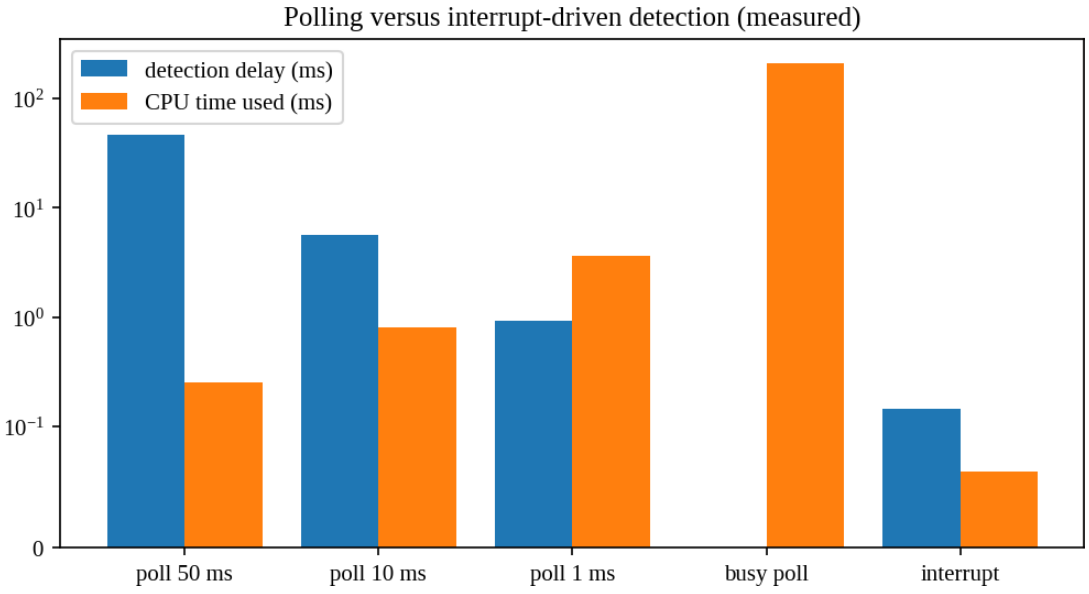


Figure 8.1 Detection delay and CPU time used: polling and interrupts (log scale)

8.6 Alternative concept: polling instead of interrupts

What the solution would be. The monitor's software loops, reading the sensor register every P milliseconds and checking whether the value is abnormal:

```
while (true) {
    v = read_sensor();
    if (v is abnormal) raise_alarm();
```

```
    sleep(P);          // or nothing, for pure busy polling
}
```

Worst-case delay. A reading that becomes abnormal just after a poll is noticed one full period later, so the worst-case delay is P plus the processing time. With the 10 ms requirement, P must be less than about 5 ms (to leave a margin for filtering and transmission). The measurements agree: polling every 50 ms noticed the event 47 ms late; every 10 ms, 5.7 ms late; every 1 ms, 0.94 ms late. With an interrupt the delay is the *interrupt latency*, the time from the signal to the first instruction of the handler: a few microseconds on a real-time kernel (the 0.146 ms measured here includes the signal delivery of a general-purpose Linux in a virtual machine). The worst-case delay of interrupts is O(1) and independent of the number of sensors; polling has worst-case delay Theta(P), and if the monitor must poll k sensors in turn it becomes O(k × per-sensor time).

CPU time and power wasted. Polling uses CPU time even when nothing happens. The CPU use is about (poll cost) / P: the measurements show 0.25 ms of CPU per 203 ms for P = 50 ms, 3.65 ms for P = 1 ms, and 209 ms (100 % of the CPU) for busy polling, compared with 0.06 ms for the interrupt design. A processor that is waiting for an interrupt can enter a sleep state (the `hlt`/`wfi` instruction) in which it uses a small fraction of the power, which matters for battery backup. Polling prevents sleep and wastes energy; polling can be better only when events are very frequent (a high-speed network device under heavy load, where interrupts would arrive too fast, which Linux's NAPI exploits by switching to polling under load).

Judgement. Interrupts are used for the monitors: they minimise worst-case delay and CPU use. Polling at a fixed rate of 5 ms or faster is only a fallback for sensors that cannot raise interrupts.

8.7 Evidence summary for Task 8

CPU modes, privileged instructions, interrupts, the timer and memory protection have been explained and related to the monitors; the measured table and graph show the trade-off between delay and CPU use for polling against interrupts.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems (4 Marks)

9.1 The levels of the memory hierarchy

Storage is organised as a hierarchy because no technology is at the same time fast, large and cheap. Going down the hierarchy, each level is slower, larger and cheaper per byte.

Level	Typical size	Typical access time	Managed by	Volatile?
Registers	about 1 KB (16-32 registers)	under 1 ns (1 cycle)	Compiler	Yes
L1 cache	32-64 KB per core	1-2 ns	Hardware	Yes
L2 cache	256 KB - 2 MB per core	3-5 ns	Hardware	Yes
L3 cache	8-64 MB shared	10-20 ns	Hardware	Yes
Main memory (DRAM)	8 GB - 4 TB	60-100 ns	OS (virtual memory)	Yes
SSD (NVMe flash)	256 GB - 8 TB	20-100 microseconds	OS and file system	No
Hard disk	1 - 20 TB	5-10 milliseconds (seek + rotation)	OS and file system	No
Network storage (SAN/NAS, cloud)	unbounded	0.2 - 100 ms	OS, storage system	No
Tape / archive	PB	seconds to minutes	Backup software	No

The ratio between neighbouring levels is 3 to 100 times, and between registers and a hard disk it is about 10 million times. The caches work because of *locality of reference*: programs tend to reuse recently used data (temporal locality) and data near recently used data (spatial locality).

Measured access times on the experimental machine

The program `t09_latency.c` follows a chain of pointers through arrays of increasing size, in random order, so each access depends on the previous one. This measures the *latency* of the level that holds the array.

Listing: t09_latency.c

```
/* Task 9: measure access time per element for working sets from 4 KB to 256 MB (poi \
      nter chasing). */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
int main(void)
{
    printf("%10s %12s\n", "size", "ns/access");
    for (size_t kb = 4; kb <= 262144; kb *= 4) {
        size_t n = kb * 1024 / sizeof(void *);
        void **a = malloc(n * sizeof(void *));
        size_t *perm = malloc(n * sizeof(size_t));
        for (size_t i = 0; i < n; i++) perm[i] = i;
        srand(1);
        for (size_t i = n - 1; i > 0; i--) { size_t j = (size_t)rand() % (i + 1); si \
      ze_t t = perm[i]; perm[i] = perm[j]; perm[j] = t; }
        for (size_t i = 0; i < n; i++) a[perm[i]] = &a[perm[(i + 1) % n]];
        void **p = &a[perm[0]];
        long steps = 20000000;
        double t0 = now();
        for (long s = 0; s < steps; s++) p = (void **)*p;
        double t1 = now();
        if (!p) return 1;
        if (kb >= 1024) printf("%7zu MB %12.2f\n", kb / 1024, (t1 - t0) * 1e9 / step \
      s);
        else printf("%7zu KB %12.2f\n", kb, (t1 - t0) * 1e9 / steps);
        free(a); free(perm);
    }
    return 0;
}
```

$ ./t09_latency

```
      size    ns/access
       4 KB         1.75
      16 KB         1.81
      64 KB         3.98
     256 KB         5.45
       1 MB         7.69
       4 MB        29.72
      16 MB        71.43
      64 MB       130.74
     256 MB       145.91
```

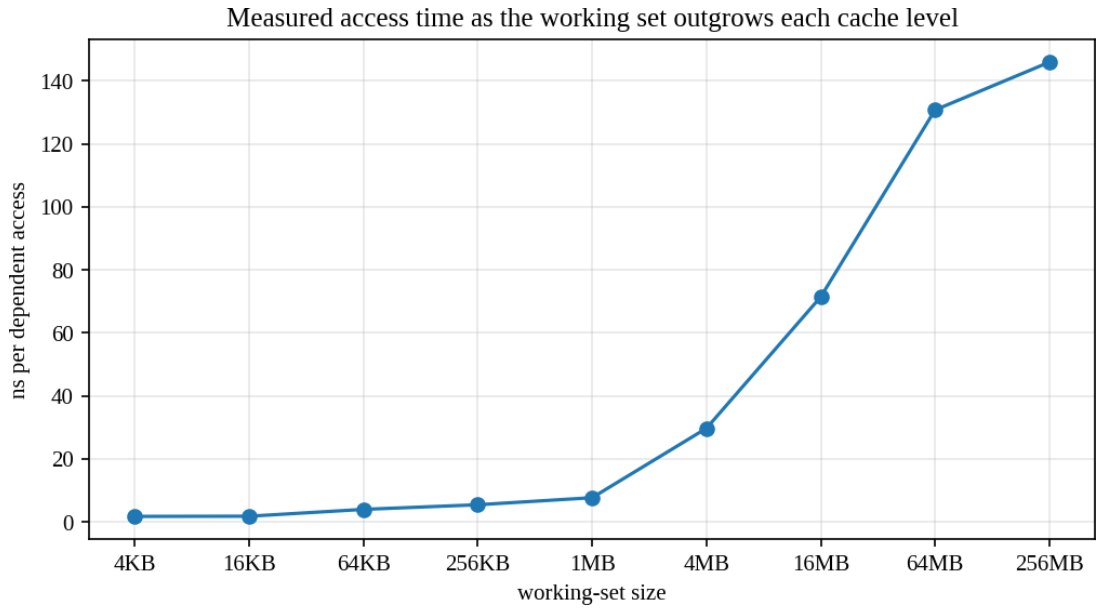


Figure 9.1 Measured access time against working-set size

The curve shows the steps: about 1.8 ns while the array fits in L1 (up to 16 KB), 4 to 5 ns for L2 (64 to 256 KB), 8 ns around 1 MB, then 30 to 70 ns for the shared L3 (4 to 16 MB) and 130 to 146 ns once the array is larger than the cache and every access goes to main memory (64 MB and above). The values were measured in a virtual machine so they are noisier than on bare metal, but the shape is the point: a program whose working set doubles beyond a cache level can slow down by a large factor.

9.2 Where should MRI images, patient records and monitor readings be kept?

Data	Properties	Placement	Reason
Live monitor readings (last few seconds)	Tiny, written 250 times per second, read by alarm logic, must never wait	Registers, cache and RAM of the monitor; a ring buffer in main memory	Needs nanosecond access; a disk access of 5 ms would break the 10 ms deadline
Monitor history (minutes to hours)	Small, sequential writes, occasionally read	Flash on the monitor, then to the central server	Flash survives power loss; sequential writes are efficient
Active patient records	Moderate size, random reads and writes, very frequent look-ups, must be durable	Database on NVMe SSDs in RAID 10, hot index and recent rows cached in RAM (buffer pool)	SSD gives 50 microsecond random access; RAM cache gives about 100 ns for hot records
Old patient records	Large, rarely read	Hard disks (RAID 6) and cloud/archive	Cheap per byte; latency of 10 ms is acceptable for rare look-ups

Data	Properties	Placement	Reason
MRI images being processed	Hundreds of MB each, read sequentially, many processors	RAM (working set) and a striped NVMe array	High bandwidth needed (several GB/s); DMA moves data without CPU
MRI images, archive	Many TB, rarely read	Hard-disk arrays and tape/cloud archive	Cost per byte
Backups	Large, write-once	Separate disk array and an off-site copy	Independence from the main system

9.3 Programmed I/O, interrupt-driven I/O and DMA

Programmed I/O (polling). The CPU executes a loop that tests the device's status register until the device is ready and then moves one word between the device register and memory. It is simple, but the CPU does nothing else during the transfer.

Interrupt-driven I/O. The CPU starts the operation and continues with other work. When the device is ready for the next word (or the operation is finished), it raises an interrupt; the CPU runs the interrupt handler, moves one word and returns. CPU time is used only when the device needs service, but each word costs one interrupt, with the overhead of saving state, running the handler and restoring state (1 to 5 microseconds).

Direct memory access (DMA). The CPU gives a DMA controller the memory address, the length and the direction, and starts the transfer. The controller moves the whole block between device and memory without the CPU, taking over the memory bus for short cycles (cycle stealing) or using bus-mastering, and raises one interrupt when the block is finished. This is the method for fast block devices (disks, network cards, scanners).

For the **MRI scanner** DMA is the right method: the data rate is very high (hundreds of MB/s), a transfer is large, and the CPUs are needed for reconstruction. Programmed I/O is acceptable only for very slow devices with little data, and interrupt-driven I/O (one interrupt per character) for keyboards and for the monitor's sensor sample stream, where each event needs prompt individual attention and the rate (hundreds per second) is low.

9.4 The effect of caching on the speed of record look-up

If a look-up in the cache takes t_c = 100 ns (memory) and a look-up that misses must go to the SSD, taking t_m = 100 µs (100,000 ns), the average time is $T = h \times t_c + (1 - h) \times t_m$, where h is the hit ratio. For h = 0.50 the average is 50.05 µs; for h = 0.90 it is 10.09 µs; for h = 0.99 it is 1.099 µs; for h = 0.999, 0.2 µs. A rise in the hit ratio from 90 % to 99 % makes look-ups nine times faster. For a hospital database in which a few active patients' records account for most accesses (strong locality), a buffer pool that holds the 5 % hottest rows

can reach a hit ratio above 95 %. The same relation applies to the operating system's page cache for files and the CPU's TLB.

9.5 Alternative concept: interrupt-driven I/O (one interrupt per word) instead of DMA

What the solution would be. The scanner raises an interrupt for every 32-bit word. Each interrupt handler reads one word from the device register and stores it in the memory buffer. Let the image be 500 MB = 524,288,000 bytes.

Number of interrupts. Words = 524,288,000 / 4 = 131,072,000 interrupts, so O(n) in the size n of the image. With DMA the controller transfers blocks of, say, 64 KB per descriptor and interrupts after each block (or once per chain of descriptors): 524,288,000 / 65,536 = 8,000 interrupts, O(n/B) for block size B (and as little as 1 if the whole image is described by a scatter-gather list).

CPU time. If one interrupt costs about 2 µs for entry, handling and return (a conservative value for modern hardware; older machines need more), the interrupt-driven transfer uses 131,072,000 × 2 µs = 262 s of CPU time, more than four minutes, for a single image, and the CPU would be completely occupied if the scanner delivers data at more than 2 MB/s (one word per 2 µs gives only 2 MB/s). The scanner needs hundreds of MB/s, so the CPU could not keep up at all and data would be lost. With DMA the CPU time is 8,000 × 5 µs (handler and set-up) = 40 ms, which is 0.015 % of the interrupt-driven cost, a ratio of about 6,500 to 1. The transfer time is then limited by the device and the bus (for 500 MB at 2 GB/s about 0.25 s).

Memory buffers. In the interrupt-driven method the device needs only a one-word (or small FIFO) buffer, because each word is moved immediately, and the final destination is the 500 MB buffer in memory. With DMA the controller writes directly into memory buffers, usually a ring of descriptors and *two or more* large buffers (double buffering, for example 2 × 1 MB) so that the processors can work on one buffer while the controller fills another; the destination must be physically contiguous or described by scatter-gather and *pinned* (locked in memory) during the transfer so that the OS does not move or page it out. The extra memory is O(B) for the descriptors and buffers, small compared with the 500 MB image.

Judgement. DMA is chosen for the MRI scanner; interrupt-driven I/O is rejected because it would take 262 s of CPU time per image and cannot sustain the required rate.

9.6 Evidence summary for Task 9

The data-placement plan (Section 9.2) and I/O method for each device are justified with typical access times (Section 9.1) and with measured latencies, and the DMA alternative has been quantified at 8,000 against 131 million interrupts.

TASK 10 – Measure and Compare (4 Marks)

10.1 Measurements planned

The assignment requires four sets of measurements: the number of system calls of the copy program for different buffer sizes; the time in user mode and kernel mode; the time to create 1,000 processes and 1,000 threads; and a comparison of the measurements with the earlier judgements. The first has been done in Task 6, the others are done here.

10.2 Number of system calls for different buffer sizes

The tracer (equivalent to `strace -c`) counted all system calls of the Task 6 program. For the 10,000-byte file the program calls read 10,001 + 21 + 4 times. The table below summarises both the program counters and the larger 4 MB file.

Buffer size b	read() calls (10,000-byte file)	read() calls (4 MB file)	Predicted ceil(n/b)+1 (4 MB)	Real time (4 MB, s)
1 byte	10,001	4,194,304	4,194,305	2.334
16 bytes	626	262,144	262,145	0.139
512 bytes	21	8,192	8,193	0.0121
4,096 bytes	4	1,024	1,025	0.0073
65,536 bytes	2	64	65	0.0062
1,048,576 bytes	2	4	5	0.0030

(The counts for 4 MB exclude the final end-of-file call in the table's measured column, since the benchmark counts only successful calls; the prediction column includes it.) The two columns agree to within one call.

10.3 Time in user mode and in kernel mode

The bash `time` keyword reports three times: *real* (wall-clock), *user* (CPU time executing the program's own code) and *sys* (CPU time executing the kernel on behalf of the program). The system-call-heavy program was timed with `time`.

```
$ bash -c 'time ./t06_bench' 2>&1 | tail -5
```

```
mmap + memcpy                       0     0.0042     0.0000     0.0013

real    0m2.406s
user    0m0.469s
sys     0m1.884s
```

The earlier benchmark table already showed the split for each method: for the 1-byte buffer the sys time was 1.85 s and the user time 0.45 s (about 80 % in the kernel), and for the 4096-byte buffer 0.0006 s of sys and 0.0030 s of user time. The kernel time dominates when there are very many tiny system calls, and falls to a small part when the buffer is large. This is the signature of system-call overhead: when a program spends most of its time in sys, reducing the *number of calls* (not the work per call) is the remedy.

10.4 Time to create 1,000 processes and 1,000 threads

The program below creates and waits for 1,000 children with `fork()`, creates and joins 1,000 threads, and also measures `posix_spawn` and `fork` + `exec`.

Listing: t10_create.c

```
/* Task 7/10: time to create and reap 1000 processes (fork, vfork-like exec-less) an \
      d 1000 threads. */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <time.h>
#include <spawn.h>
#include <sys/wait.h>
#include <sys/mman.h>
extern char **environ;
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); ret \
      urn t.tv_sec + t.tv_nsec / 1e9; }
static void *thr(void *a) { (void)a; return NULL; }
int main(int argc, char **argv)
{
    size_t heap_mb = argc > 1 ? (size_t)atoi(argv[1]) : 0;
    char *heap = NULL;
    if (heap_mb) { heap = malloc(heap_mb << 20); for (size_t i = 0; i < (heap_mb <<  \
      20); i += 4096) heap[i] = 1; }
    const int N = 1000;
    double t0 = now();
    for (int i = 0; i < N; i++) { pid_t p = fork(); if (p == 0) _exit(0); waitpid(p, \
       NULL, 0); }
    double t1 = now();
    for (int i = 0; i < N; i++) { pthread_t t; pthread_create(&t, NULL, thr, NULL);  \
      pthread_join(t, NULL); }
    double t2 = now();
```

```
    for (int i = 0; i < N; i++) { pid_t p; char *av[] = {"true", NULL}; posix_spawnp \
      (&p, "true", NULL, NULL, av, environ); waitpid(p, NULL, 0); }
    double t3 = now();
    for (int i = 0; i < N; i++) { pid_t p = fork(); if (p == 0) { execlp("true", "tr \
      ue", (char *)NULL); _exit(127); } waitpid(p, NULL, 0); }
    double t4 = now();
    printf("parent heap touched: %zu MB\n", heap_mb);
    printf("1000 x fork+exit+wait      : %8.1f ms  (%6.1f us each)\n", (t1 - t0) * 1 \
      e3, (t1 - t0) * 1e6 / N);
    printf("1000 x thread create+join  : %8.1f ms  (%6.1f us each)\n", (t2 - t1) * 1 \
      e3, (t2 - t1) * 1e6 / N);
    printf("1000 x posix_spawn(true)   : %8.1f ms  (%6.1f us each)\n", (t3 - t2) * 1 \
      e3, (t3 - t2) * 1e6 / N);
    printf("1000 x fork+exec(true)     : %8.1f ms  (%6.1f us each)\n", (t4 - t3) * 1 \
      e3, (t4 - t3) * 1e6 / N);
    return 0;
}
```

$ ./t10_create

```
parent heap touched: 0 MB
1000 x fork+exit+wait     :     59.5 ms  (  59.5 us each)
1000 x thread create+join  :    10.3 ms  (  10.3 us each)
1000 x posix_spawn(true)   :   681.4 ms  ( 681.4 us each)
1000 x fork+exec(true)     :   880.5 ms  ( 880.5 us each)
```

The same program was run again under the shell's `time` keyword to see the user and system CPU time (a separate run, so the individual values differ slightly from the table above, as expected from timing noise):

$ bash -c 'time ./t10_create' 2>&1 | tail -4

```
real    0m1.542s
user    0m1.349s
sys     0m0.152s
```

$ ./t10_create 200

```
parent heap touched: 200 MB
1000 x fork+exit+wait     :   2120.9 ms  (2120.9 us each)
1000 x thread create+join  :    13.8 ms  (  13.8 us each)
1000 x posix_spawn(true)   :   869.1 ms  ( 869.1 us each)
1000 x fork+exec(true)     :   3681.9 ms  (3681.9 us each)
```

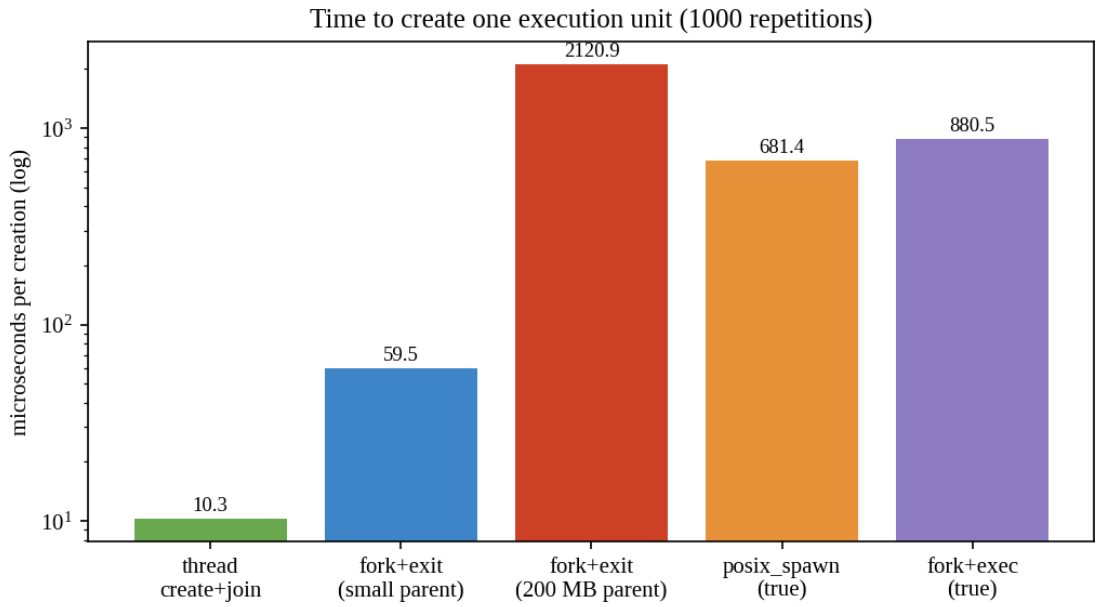


Figure 10.1 Time to create one process or thread (log scale)

Results and comparison with expectations

Measurement	Result	Expectation	Verdict
Thread creation (create + join)	10.3 microseconds	Much cheaper than a process: no new address space	Confirmed
Process creation by fork (small parent)	59.5 microseconds	Several times a thread	Confirmed (5.8 times)
Process creation by fork with a 200 MB parent	2,121 microseconds	Cost grows with page-table size	Confirmed (35 times a small fork)
Thread creation with a 200 MB parent	13.8 microseconds	Independent of the parent's memory size	Confirmed
fork + exec of a small program	880 microseconds	Larger than fork alone because the loader runs	Confirmed
posix_spawn	681 microseconds (869 for 200 MB parent)	Cheaper than fork + exec for a large parent	Confirmed

10.5 How the measurements support or contradict the earlier judgements

- **Supported:** a system call is expensive; a 4,096-byte or larger buffer removes most of the overhead. The recommended backup buffer (256 KB) is consistent with the flat part of the curve.
- **Supported:** interrupts give a smaller delay and a lower CPU use than polling (Task 8).

- **Supported:** threads are much cheaper to create than processes, but processes give isolation (Task 7).
- **Qualified:** the thread-per-task design for a monitor would not give a latency benefit alone. The cost difference of about 50 µs between a process and a thread is far below the 10 ms deadline, so the choice between them for the monitor should be made on *isolation* (fault containment), not on speed.
- **Limitations:** the machine is virtualised, has one CPU and noisy timing, so absolute times are not those of a real monitor. The figures give ratios and shapes, and the design decisions rely on those.

10.5 a Memory used by processes and threads

A thread shares the address space, heap, code, global data and open files with the other threads of its process and has only its own stack (8 MB virtual by default, of which only the touched pages, a few KB, are resident), its registers and a thread-control block. A process created by `fork` initially shares all pages with its parent copy-on-write; its private memory is the page tables (about 8 bytes per 4 KB page, so 400 KB of page-table entries for 200 MB), the kernel process structure and kernel stack (about 10 to 20 KB), plus any page it writes. Therefore 1,000 threads cost roughly 1,000 × (stack touched, 8 to 16 KB, plus about 2 KB structures) ~ 10 to 20 MB, and 1,000 forked children of a small process cost similar amounts initially, but each child that modifies its data grows by whole pages; a child of a 200 MB parent needs up to 400 KB for page tables in addition.

10.6 Alternative concept: threads instead of processes for the measurements

What the result would be. If threads were created instead of processes (as in the second row of the table), 1,000 threads were created and joined in 10.3 ms against 59.5 ms for 1,000 processes (small parent) and 2,121 ms for 1,000 processes (200 MB parent).

Complexity. Process creation costs O(1) + O(P) where P is the number of mapped pages of the parent (page-table copy); thread creation is O(1). Memory for k children: processes O(k × (kernel structures + private pages)) and threads O(k × stack).

Explaining the difference by what is shared. A thread shares the address space, so no page tables, memory map or file table needs to be copied. A process needs its own page tables (copy-on-write marking of every writable page of the parent, which is what makes the cost proportional to the size of the parent), a new file-descriptor table, signal handlers, credentials, and a TLB flush on a context switch between address spaces. Sharing is also the reason for the weakness of threads: a bug in one thread can corrupt the memory of all, and shared data must be protected with locks.

10.7 Evidence summary for Task 10

The measured tables and graphs, the comparison with the expected results and the explanation of the differences are given above.

TASK 11 – Recommend and Defend the Operating-System Plan (4 Marks)

11.1 The recommended plan

Figure 1.1 Proposed heterogeneous operating-system plan for the 500-bed hospital

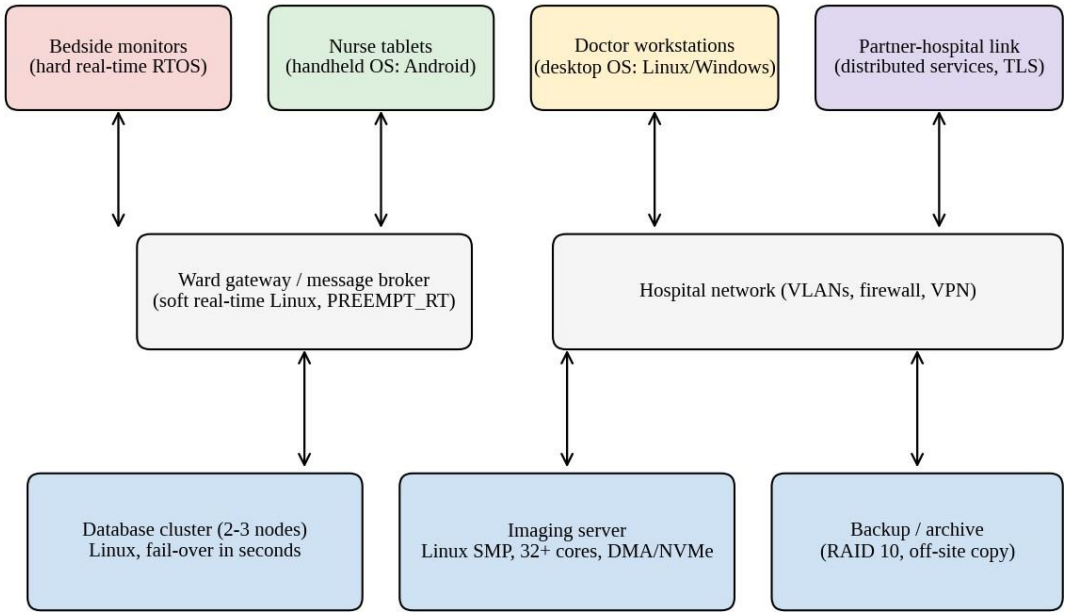


Figure 11.1 The recommended operating-system plan for the hospital

Environment	OS type	Structure	Main services kept	Protection	I/O method
Bedside monitor	Hard real-time embedded OS (QNX-class microkernel RTOS, or Zephyr/FreeRTOS with MPU on small devices)	Microkernel (or small monolithic with MPU)	Scheduler, interrupts, sensor drivers, secure network client, watchdog	Signed firmware, secure boot, memory protection between tasks, TLS to gateway	Interrupt-driven for sensors; DMA for the display and network

Environment	OS type	Structure	Main services kept	Protection	I/O method
Nurse tablet	Handheld OS (Android Enterprise or iPadOS) under mobile-device management	Layered on a modular Linux kernel	Touch UI, app sandbox, Wi-Fi, barcode/camera, power management	Disk encryption, screen lock, per-app sandbox, remote wipe, VPN	Interrupt-driven, DMA on camera and flash
Doctor workstation	Desktop OS (Windows or Linux desktop) managed centrally	Hybrid (Windows) or modular monolithic (Linux)	GUI, EHR client, imaging viewer, printing	User accounts from directory service, ACLs, disk encryption, patching	Interrupt-driven and DMA
Patient database	Clustered Linux server OS with fail-over	Modular monolithic	Journaling file system, RAID, replication, resource control, audit	Mandatory access control (SELinux), encrypted volumes, audit log	DMA with NVMe and network, asynchronous I/O
Imaging server	Linux SMP (NUMA-aware)	Modular monolithic	Large memory, scheduler tuned for batch, huge pages, striped storage	Isolated network segment, ACLs	DMA from scanner and NVMe
Partner link	Linux gateway servers (distributed services)	Modular monolithic	TLS/VPN, message queue with retries, identity federation	Mutual authentication, encryption in transit, audit	Interrupt-driven and DMA on NICs

The plan uses *three operating-system families*: an embedded real-time OS for the devices that have deadlines, a handheld OS for tablets, and mainstream Linux (with a Windows or Linux desktop for workstations) for everything else. Between the monitors and the rest of the hospital sits a *ward gateway* running a soft real-time Linux with the PREEMPT_RT patches, which receives the monitors' messages and forwards them, so that the monitors never depend on the general network.

11.2 Why this plan satisfies the clinical concerns

1. **Alarms delayed?** The 10 ms alarm path stays inside the monitor on a hard real-time kernel with interrupt-driven I/O (Tasks 2, 3, 8). The worst-case alarm delay is bounded by the interrupt latency (microseconds) rather than by load. The

measurement in Task 8 shows that polling is slower or wasteful and that interrupts give the smallest delay.

2. **Database stopping during updates?** The two-node cluster is updated one node at a time with a fail-over of seconds (Task 2), and the journaling file system keeps the data consistent after a failure (Task 4).
3. **Tablet batteries?** The handheld OS has power management and a small footprint (Task 4).

11.3 Limitations and risks of the plan

- **Heterogeneity.** Three OS families mean three update procedures, three sets of vulnerabilities and more skills needed. Mitigation: central device management, a software bill of materials, automated patch pipelines and staging tests.
- **Gateway as a single point of failure.** Each ward needs two gateways in an active/standby pair.
- **Real-time claims depend on configuration.** Soft real-time Linux can still show latency spikes (long driver code, power-saving states); it must be validated with latency tests (for example cyclictest) under load before being used for any alarm path.
- **Certification.** Medical devices must meet regulatory standards (IEC 62304 for software, IEC 60601); an RTOS with certification evidence reduces effort, but the hospital depends on the vendor.
- **Measurement limits.** The measurements here were made on a one-core virtual machine, not the target hardware; they support the ratios, but the real latency figures must be re-measured on the monitor hardware.
- **Cost.** Clustered servers, certified RTOS licences and device management cost more than one general OS, but this is justified by the safety requirements.

11.4 Does the plan remain suitable for a group of three hospitals?

Mostly yes. The monitors and tablets are *per device* and do not change with the number of hospitals. The database cluster scales to three sites by adding replicas and by partitioning records by hospital (sharding), using asynchronous replication between sites to avoid paying the wide-area round trip on every write. The partner link becomes a federation of identity and record-sharing services. What must change is the administration: with about 1,500 monitors and 360 tablets, manual management is impossible, so automated configuration management (infrastructure as code), central logging and monitoring, and a disaster-recovery site become essential. The distributed-file/records service must be designed to tolerate the loss of a whole hospital site.

11.5 Alternative concept: virtual machines on two large servers

What the solution would be. All server roles (database, imaging, gateway, workstation desktops as virtual desktops) run as virtual machines on two large physical hosts under a type-1 hypervisor such as KVM or ESXi, instead of separate physical machines.

Response time for the monitors. The monitors themselves stay on their own hardware. If the *gateway* is a virtual machine, its interrupts are delivered through the hypervisor and the guest scheduler is itself scheduled by the host scheduler, which adds latency and jitter: typically tens of microseconds on a tuned host, but occasionally milliseconds when the host is overloaded or when the VM is descheduled. For the alarm path this is acceptable only if the VM has pinned CPUs and real-time priority; otherwise the 10 ms bound cannot be guaranteed. Hence the hard real-time loop must not run in a shared VM.

Memory and processor overhead. CPU-bound work runs at 95 to 99 % of native speed with hardware virtualisation (VT-x/AMD-V); I/O-bound work suffers 5 to 20 % unless paravirtual drivers (virtio) or SR-IOV pass-through are used. Each VM needs its own guest kernel and page tables: 0.5 to 2 GB per VM. For ten VMs that is 5 to 20 GB extra, i.e. memory overhead O(V × guest footprint) for V virtual machines, unless the host shares identical pages. Nested paging makes a TLB miss up to 24 memory references instead of 4, which affects large-memory workloads such as imaging unless huge pages are used.

Reliability. Consolidation gives live migration, snapshots and fast restart, which is a big benefit; but two hosts are now the points of failure for everything. If one host fails, the other must carry the full load, so each must be sized at 100 % of the workload (N+1 with N = 1), and a hypervisor bug or a storage failure can take down all the VMs on it. A shared failure domain is the opposite of the independence that the plan needs for the database nodes: two database VMs on the same host provide no fail-over at all for a host crash, so the primary and standby must be on different hosts.

Judgement. Virtualisation is adopted for the workstations (virtual desktops), the partner gateways and development/test, and for imaging only with pass-through and huge pages. The database cluster and the monitors remain on dedicated hardware, which retains the original plan.

11.6 Evidence summary for Task 11

The plan is supported by the analysis in Tasks 1 to 10, with limits and risks stated, and it scales to three hospitals with automation.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

Changed requirement. The hospital adds remote monitoring devices in patients' homes that send readings over the internet, and moves patient records to a cloud service shared with the partner hospitals. The original plan is re-evaluated, and the decision is to **MODIFY** it: retain the plan for the hospital core and extend it with new device and cloud tiers, rather than redesign everything.

A. Impact Analysis (2 Marks)

Which operating-system types are affected

Component	Type in original plan	Affected by the change?	How
Bedside monitor	Hard real-time RTOS	Not directly; they stay on the hospital network	But they must be able to send data to the cloud through the gateway
Home monitoring device	Did not exist	New environment	Needs an embedded real-time OS with a secure network stack, remote update and long battery or mains-backup life
Nurse tablets, workstations	Handheld, desktop	Partly	They now access records through the cloud (browser/API) rather than only the local database
Local database cluster	Clustered Linux	Strongly	Records move to the cloud; local cluster becomes a cache/replica or a hot standby, or is retired for records
Partner link	Distributed services	Strongly	Replaced by a shared cloud service with federated identity
Cloud service	Not in the plan	New	Linux virtual machines or containers, orchestrated, replicated across zones

Which services are affected

- **Communication** becomes the dominant service: devices now connect over an untrusted, slow, intermittent network (home broadband, mobile data) instead of a hospital LAN. The OS must provide a robust TCP/IP and TLS stack, retry with backoff, store-and-forward of readings when the network is down, and time synchronisation.

- **File system** on the home device: a small log in flash with wear levelling, so that readings survive power failure while offline.
- **Resource allocation and power management**: the home device is battery- or mains-powered with backup; the OS must handle sleep states without missing deadlines.
- **Error detection**: watchdog, self-test, and *remote health reporting* now become essential because no technician is nearby.
- **Accounting and audit** become more important: the cloud must record who accessed which record from which partner hospital.
- **User interface**: the home device needs a very simple interface for an elderly patient (an alarm button and a status light), not a full GUI.

Which protection mechanisms are affected

The security perimeter has moved. In the original plan the hospital LAN was a trusted zone protected by a firewall; now readings travel over the internet and records live on shared infrastructure owned by a cloud provider. Affected mechanisms: **authentication** (every device needs its own identity and certificate; staff need multi-factor authentication; partner hospitals need federated identity), **encryption** (TLS 1.3 in transit, encryption at rest with keys held by the hospital), **authorisation** (role- and attribute-based access control; a partner hospital sees only the patients it treats), **update integrity** (signed firmware over the air), **isolation in the cloud** (tenant separation, virtual-machine and container isolation), and **audit** (immutable logs). The memory protection and user/kernel mode mechanisms of Task 8 still apply on each device, but they are no longer sufficient alone: the main risk is now network attack and misconfiguration.

B. Evaluation of the Original Design (2 Marks)

The question is whether the original plan can support home devices and cloud records. The evaluation uses three criteria from the assignment.

Response time. The original alarm path is inside the monitor and on the local network, with a worst-case alarm delay bounded by the interrupt latency. This property *does not depend on the internet*, so the local alarm in the home device can be retained if the device follows the same design (hard real-time local alarm). What cannot be guaranteed is the *delivery of the alarm to the hospital*: internet round trips vary from 20 ms to several seconds and links fail. The original 10 ms requirement applies to the *local detection and local alarm*; remote notification needs a separate, looser objective (for example, delivery within 5 seconds with 99.9 % probability, and a fallback through a second network path such as mobile data). The original plan has no such mechanism, so it is **insufficient** for this

requirement.

Reliability. The cluster with fail-over gave high availability inside one site. A cloud service gives availability across zones, but the hospital now depends on the internet connection and on the provider. The original plan has no design for loss of connectivity: if the line to the cloud fails, nurses cannot reach records. The original plan is **partly sufficient** for reliability: the cluster design transfers to the cloud as a replicated service, but needs a local fallback.

Security. The original design trusted the hospital network and used OS-level protection. With home devices and cloud records, the attack surface is thousands of devices on unmanaged home networks and an internet-facing service. The original design is **insufficient** here without a new identity, encryption and update infrastructure.

Criterion	Original plan	Supports changed requirement?	Gap
Local alarm latency	Hard real-time RTOS, interrupt-driven	Yes (reusable in home device)	None for local alarm
Remote alarm delivery	Wired LAN to gateway	No	Needs store-and-forward, two network paths, acknowledged delivery
Availability of records	Two-node cluster in one site	Partly	Needs multi-zone cloud replica plus local cache for outage
Security	Perimeter firewall, OS permissions	No	Needs per-device identity, TLS everywhere, signed updates, zero-trust access
Administration	Mixed fleet of 700 managed devices	No	Needs remote device management for thousands of unmanaged homes
Scalability	1,500 monitors at most	Partly	Cloud scales elastically; devices need automated provisioning

Evidence from the earlier tasks supports this evaluation: Task 5 and Task 6 showed that the OS interface and its cost can be controlled, Task 8 showed that interrupt-driven design gives bounded local latency, and Task 11 identified administration and the gateway as risks. Task 11's own conclusion that the plan scales to three hospitals only with automation applies even more strongly to thousands of home devices.

C. Adaptation or Redesign (2 Marks)

Figure 12.1 Revised design with home monitoring and cloud records

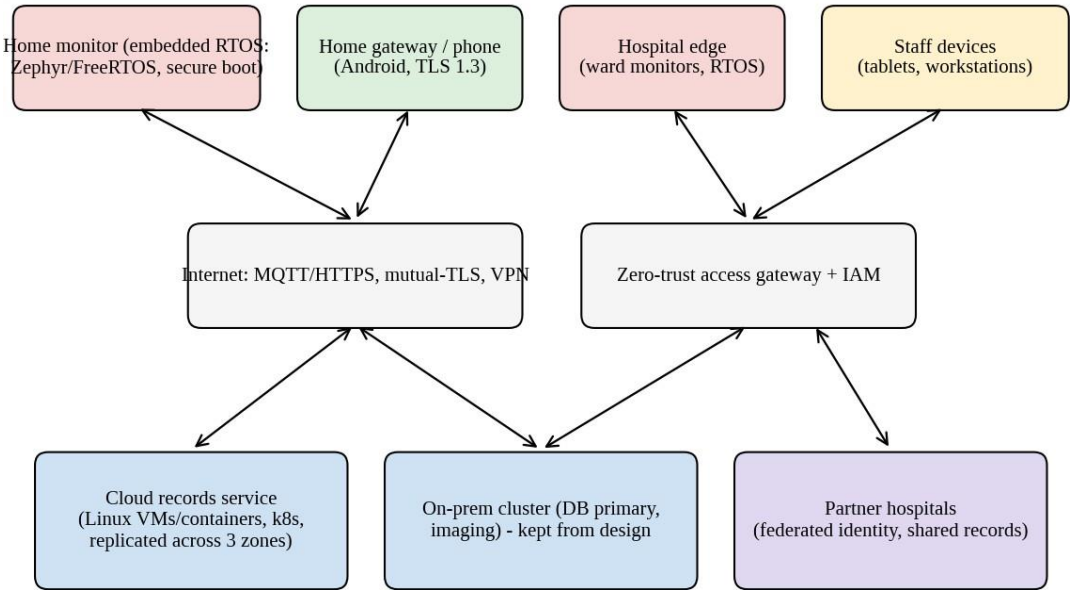


Figure 12.1 Revised design: home devices, cloud records and the hospital core

The decision is to **modify** the design. What stays unchanged: the hard real-time RTOS on the bedside monitors, the handheld OS on the tablets, the desktop OS on the workstations, the SMP imaging server, and the principle of using different OS types for different requirements. What changes is described below.

1. **Home monitoring device: embedded real-time OS.** The device runs an RTOS (Zephyr, FreeRTOS with MPU, or a certified QNX-class OS) with the same architecture as the bedside monitor: interrupt-driven acquisition and a *local* alarm that is raised within the same 10 ms bound without depending on the internet. In addition, it has a TLS 1.3 client with a device certificate stored in a secure element, an MQTT or HTTPS reporting task at lower priority, a flash log with store-and-forward for the offline case, a watchdog, and over-the-air signed updates with rollback. The OS structure is a microkernel RTOS (or MPU-protected tasks), so that the network task cannot corrupt the alarm task.
2. **Home gateway or patient's phone.** Where a device has no direct internet access, it connects over Bluetooth or Wi-Fi to a gateway (a handheld OS app or a small Linux box). The gateway keeps a second network path (mobile data) and forwards readings.
3. **Cloud records service: distributed system of Linux nodes.** The records move to a cloud service built on Linux virtual machines or containers (a clustered system at larger scale), replicated across at least three availability zones, with a managed database offering fail-over, encrypted storage, and automatic scaling. This replaces the single-site cluster and the partner link; partner hospitals access the same service

through federated identity, so the earlier point-to-point links are no longer needed.

4. **Hospital core.** The local database cluster is *retained* as a read replica and a local cache so that wards can still read the records of current patients during an internet outage (and the clinical staff's concern about availability is met in both directions). The imaging server stays local because of the data volume (500 MB per scan), and only reports and selected images go to the cloud.
5. **Security architecture.** Zero-trust access: every device and person authenticates for every access; mutual TLS between services; role- and attribute-based access control; hospital-controlled encryption keys; hardware-backed device identity; central audit logs; and an automated vulnerability and patch pipeline for Linux, the RTOS and the tablets.
6. **Administration.** Device management service (over-the-air updates, health reporting), infrastructure as code for the cloud, and monitoring dashboards.

D. Evidence-Based Defence (2 Marks)

Why modify and not retain or redesign. *Retain* is not defensible because the evaluation in part B found gaps in remote alarm delivery, security and administration, and the clinical staff's three concerns (alarm delay, database stoppage, battery life) would be joined by new ones (internet outage, data exposure). *Redesign everything* is not justified either: the measurements and the analysis show that the core principle, matching the OS type to the requirement, remains correct. Interrupt-driven real-time design on the device (Task 8) is independent of the network, the system-call cost analysis and the buffer-size result (Tasks 6 and 10) still govern the performance of the Linux servers, and the process-versus-thread measurements (Task 10) still justify using isolated processes for services in the cloud. Therefore the sensible decision is to keep what is proven and add what is new.

Evidence and numbers.

- Local alarm: the interrupt-driven design measured a delay of 0.146 ms against 47 ms for 50 ms polling and 5.7 ms for 10 ms polling (Task 8). The local alarm in the home device is retained with the same mechanism and therefore keeps its worst-case bound whatever the internet does.
- Remote delivery: with an internet round trip of up to 1 s and a retry interval doubling from 1 s, five attempts cover a 31 s outage; two network paths with independent failure probability 1 % each reduce the chance that both are down to 0.01 % (0.01 × 0.01), which meets a 99.9 % delivery target.
- Availability: a record replicated in three zones, each with 99.9 % availability and independent failures, is unavailable with probability 0.001^3 = 10■■ in theory; in

practice correlated failures (a bad software update, a shared control plane) dominate, so a local replica adds independent protection.

- Cost of syscalls and I/O on the cloud servers: the same buffer-size lesson holds: 256 KB buffers for replication streams and backups.

Trade-offs and remaining risks.

Decision	Benefit	Cost or risk	Mitigation
Embedded RTOS on home devices	Local alarm independent of network	Certification effort, fleet management of thousands	Use a certified OS; OTA updates; device registry
Cloud records	Elastic, replicated, shared with partners	Dependence on the provider and the internet; data-protection law; ongoing cost	Local read replica; hospital-held encryption keys; contract and exit plan
Keeping a local replica	Works during internet outage	Duplicate cost; replication complexity; consistency lag	Asynchronous replication with monitoring of the lag
Zero-trust security	Stronger protection on untrusted networks	More complex; may slow clinicians	Single sign-on; break-glass emergency access with audit
Federated identity for partners	No separate accounts; revocation is central	Trust in partner identity providers	Contractual security baseline; attribute-based limits

Final decision. The hospital should **modify** its operating-system plan: keep the heterogeneous, requirement-driven design for the hospital core; add an embedded real-time OS with secure, store-and-forward networking for home devices; replace the single-site cluster and point-to-point partner link by a replicated cloud records service with a retained local replica; and replace the trusted-perimeter security model by zero-trust authentication, encryption and signed updates. This decision integrates the evidence of Tasks 1 to 11 with the changed requirements and keeps the clinical safety properties: bounded local alarm latency, a database that is always available, and devices that last.

13. Conclusion and Summary of Evidence

The assignment asked for an evaluation of the operating-system design of a smart hospital, with judgements defended by analysis and measurement. The main conclusions are the following.

1. A single general-purpose operating system cannot guarantee the 10 ms alarm deadline, uses too much memory and power on small devices, and spreads the damage of a faulty update across the whole hospital. Different environments need different

operating-system types: hard real-time for monitors, handheld for tablets, desktop for workstations, a cluster for the database, SMP for imaging, and distributed services for sharing.

2. The structure of the OS decides how a fault spreads. A microkernel or memory-protected RTOS is best for the monitors; a modular monolithic kernel is best for the database server.
3. System calls are the gateway to OS services and are expensive. The measurements show that copying 4 MB with a 1-byte buffer needs 4.2 million calls and 2.3 s, but with a 64 KB buffer only 64 calls and 6 ms. Library buffering gives most of this benefit automatically.
4. Process control with fork, exec and wait gives flexibility and isolation. Threads are about six times cheaper to create than small processes and 200 times cheaper than a fork of a 200 MB process, but share a fate.
5. Interrupts and DMA make the CPU available for useful work; polling wastes CPU, and interrupt-driven transfer of a large image costs about 6,500 times more CPU time than DMA.
6. When the requirements change (home devices, cloud records), the design is modified, not discarded, because the reasoning method that matched the OS to the requirement remains valid.

Programs and files produced

Program	Task	Purpose
t05_demo.c, mystrace.c, t05_stdio.c	5	System-call trace and library buffering
t06_copy.c (completed code)	6	Copy with read/write and count calls
t06_bench.c	6, 10	Timing of buffer sizes, stdio and mmap
t07_shell.c (completed code), t07_noflush.c, t07_zombie.c	7	fork/exec/wait, buffer duplication, zombie
t08_polling.c	8	Polling versus interrupt-style detection
t09_latency.c	9	Memory-hierarchy latency
t10_create.c	10, 7	Creation cost of processes, threads and spawn